“Smart Leave Management System”

A Database Systems Engineering and Distributed Backend Development Project

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

&

DEPARTEMENT OD COMPUTER SECIENCE AND INFORMATION TECHNOLOGY

By

RAYANKI HIMAVARSHINI	**2520030141**
GURRAPU NIKSHITHA	**2520030512**
VEMUA NAGA VENKATA SAI CHANDU	**2520030522**

Under the Esteemed Guidance of

Dr. Spandana

Assistant Professor

Department of Computer Science and Engineering



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

&

DEPARTMENTOFCOMPUTERSCIENCEANDINFORMATIONTECHNOL



CERTIFICATE

This is to certify that the project based report entitled "Smart Leave Management System" is a bonafide work done and submitted by **RAYANKI HIMAVARSHINI – 2520030141**, **GURRAPU NIKSHITHA – 2520030512** and **VEMUA NAGA VENKATA SAI CHANDU – 2520030522** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in the Department of Computer Science and Engineering, K L (Deemed to be) University, during the academic year **[TO BE PROVIDED]**.

Signature of the Guide
Dr. Spandana
Assistant Professor, Department of Computer Science and Engineering

Signature of the Course Coordinator
[TO BE PROVIDED]

Signature of the HOD
[TO BE PROVIDED]

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

DECLARATION

The Project Report entitled " Smart Leave Management System" is a record of bonafide work of

RAYANKI HIMAVARSHINI – 2520030141, **GURRAPU NIKSHITHA – 2520030512** and **VEMUA NAGA VENKATA SAI CHANDU – 2520030522**, submitted in partial fulfillment for the award of B.Tech in Computer Science and Engineering to the K L (Deemed to be) University. The results embodied in this report have not been copied from any other department, University or Institute.

RAYANKI HIMAVARSHINI – 2520030141

GURRAPU NIKSHITHA – 2520030512

VEMUA NAGA VENKATA SAI CHANDU – 2520030522

ACKNOWLEDGEMENT

The success of this project would not have been possible without the timely help and guidance rendered by many people. We wish to express our sincere thanks to all those who have assisted us in one way or the other in the completion of our project, *Ticket Booking System*.

Our greatest appreciation goes to our guide, **Dr. Spandana**, Assistant Professor, Department of Computer Science and Engineering, for her tremendous support, encouragement and guidance throughout this project. We also thank our Course Coordinator, **[TO BE PROVIDED]**, for the continuous support extended to us.

We express our gratitude to **[TO BE PROVIDED]**, Head of the Department of Computer Science and Engineering, for providing us with adequate facilities, ways and means by which we were able to complete this project.

We thank all the members of the teaching and non-teaching staff, and everyone who has assisted us directly or indirectly in the successful completion of this project. Finally, we sincerely thank our parents, friends and classmates for their kind help and cooperation during our work.

RAYANKI HIMAVARSHINI – 2520030141

GURRAPU NIKSHITHA – 2520030512

VEMUA NAGA VENKATA SAI CHANDU – 2520030522

ABSTRACT

Booking tickets for movies, events and shows through the internet has become a routine activity, but the convenience of online booking brings a difficult engineering problem: many users may try to reserve the same seat at the same moment. If such requests are not controlled, two users may both be told that they own the same seat, which damages the reliability of the system and the trust of its users. This project, *Ticket Booking System*, addresses that problem through a web-based application that combines a relational database with distributed locking and message queuing.

The system allows a user to register and log in, browse shows, select a date and time, view real-time seat availability, select seats, make payment, receive a booking ID and digital ticket, view booking history, and cancel bookings. The frontend is built with React.js, and the backend is developed with Java and Spring Boot. MySQL stores users, shows, seats, bookings and the related status information. Redis is used for distributed seat locking, so that when one user selects a seat a temporary lock is placed on it and a second user attempting the same seat receives a conflict response. RabbitMQ is used to queue booking, payment and refund-related requests so that they are processed in an orderly manner. After a booking is successfully confirmed, the seat becomes BOOKED and the Redis lock is released; cancellation of a booking initiates refund processing.

The project is built with Maven, tested through Postman, and developed in Visual Studio Code, and it runs on Windows or Linux with the Chrome or Edge browser. The report presents the analysis, design, implementation and testing of the system, together with its limitations and future scope, which includes a mobile application, cloud deployment, a real payment gateway, email/SMS notifications, analytics and AI recommendations, support for multiple theatres, and stronger security.

Keywords: Ticket Booking, Concurrent Seat Booking, Distributed Locking, Redis, RabbitMQ, Spring Boot, MySQL, React.js.

TABLE OF CONTENTS

Acknowledgement **4**

Abstract **5**

List of Abbreviations **12**

Chapter 1 – Introduction **13**

1.1 Introduction 13

1.2 Background 13

1.3 Problem Statement 14

1.4 Existing System 14

1.5 Limitations of Existing System 14

1.6 Proposed System 15

1.7 Advantages of Proposed System 15

1.8 Objectives 16

1.9 Scope of the Project 16

1.10 Organization of the Report 17

Project Team and Responsibilities 17

Chapter 2 – Literature Survey / Related Work **18**

2.1 Introduction 18

2.2 Existing Ticket Booking Approaches 18

2.3 Online Ticket Booking Systems 18

2.4 Concurrent Booking Problems 19

2.5 Distributed Locking 19

2.6 Message Queuing 19

2.7 Database Management in Booking Systems 20

2.8 Summary of Literature Survey 20

Chapter 3 – System Analysis **21**

3.1 Problem Analysis 21

3.2 Functional Requirements 21

3.3 Non-Functional Requirements 22

3.4 Hardware Requirements 22

3.5 Software Requirements 22
3.6 User Requirements 23
3.7 System Constraints 23
3.8 Feasibility Analysis 23
3.8.1 Technical Feasibility 23
3.8.2 Economic Feasibility 23
3.8.3 Operational Feasibility 24

Chapter 4 – System Design 25
4.1 System Architecture 25
4.2 Architecture Components 25
4.3 Frontend Layer 26
4.4 Backend Layer 26
4.5 Database Layer 26
4.6 Redis Distributed Locking Layer 26
4.7 RabbitMQ Message Queue Layer 27
4.8 User Module 27
4.9 Show and Seat Module 27
4.10 Booking and Payment Module 27
4.11 Concurrency and Queue Module 27
4.12 Admin Module 27
4.13 Data Flow Diagram 28
4.14 Use Case Diagram 28
4.15 Activity Diagram 29
4.16 Sequence Diagram 30

4.17 Class Diagram 31
4.18 Entity Relationship Diagram 32
4.19 Database Design 33

Chapter 5 – Implementation 34
5.1 Implementation Overview 34
5.2 Frontend Implementation using React 34
5.3 Backend Implementation using Java and Spring Boot 34
5.4 REST API Implementation 34
5.5 MySQL Database Implementation 35
5.6 Redis Seat Lock Implementation 35

5.7 RabbitMQ Queue Implementation 35
5.8 User Registration and Login 36
5.9 Show Browsing 36
5.10 Seat Selection 36
5.11 Booking Process 36
5.12 Payment Process 37
5.13 Booking Confirmation 37
5.14 Digital Ticket Generation 37
5.15 Cancellation and Refund 37
5.16 Admin Operations 37
5.17 Concurrency Handling 38

Chapter 6 – Testing 40

6.1 Testing Introduction 40
6.2 Testing Strategy 40
6.3 Unit Testing 40
6.4 Integration Testing 40
6.5 API Testing 41
6.6 Functional Testing 41
6.7 Database Testing 41
6.8 Redis Lock Testing 41
6.9 RabbitMQ Testing 41
6.10 Concurrency Testing 41
6.11 Test Cases 42

Chapter 7 – Results and Discussion 44

7.1 System Results 44
7.2 User Registration/Login Results 44
7.3 Show Browsing Results 44
7.4 Seat Selection Results 45
7.5 Booking Results 45
7.6 Payment Results 45
7.7 Cancellation/Refund Results 45
7.8 Redis Concurrency Results 45
7.9 RabbitMQ Queue Results 45
7.10 Admin Results 45
7.11 Discussion 46

Chapter 8 – Application Screenshots 47

Chapter 9 – Conclusion and Future Scope 52

9.1 Conclusion 52

9.2 Achievements 52

9.3 Limitations 52

9.4 Future Scope 53

References 54

Appendices 55

Appendix A – Important Source Code 55

Appendix B – API Endpoints 55

Appendix C – Database Tables 55

Appendix D – Additional Screenshots 56

Appendix E – Test Cases 57

LIST OF FIGURES

Figure 4.1 High-level architecture of the Smart Leave Management System 25

Figure 4.2 Level-1 Data Flow Diagram 28

Figure 4.3 Use case diagram of the Smart Leave Management System 29

Figure 4.4 Activity diagram of the seat booking process 30

Figure 4.5 Sequence diagram of a successful booking 31

Figure 4.6 Conceptual class diagram 32

Figure 4.7 Conceptual entity relationship diagram 33

Figure 5.1 Queue processing through RabbitMQ (names of queues to be confirmed) 36

Figure 5.2 Two users attempting to book the same seat 38

Figure 5.3 Seat states during the booking process 38

Figure 8.1 Login Page 47

Figure 8.2 Registration Page 47

Figure 8.3 Show Listing Page 48

Figure 8.4 Seat Selection Page 48

Figure 8.5 Booking Page 48

Figure 8.6 Payment Page 49

Figure 8.7 Digital Ticket 49

Figure 8.8 My Bookings 49

Figure 8.9 Cancellation/Refund Status 50

Figure 8.10 Admin Dashboard 50

Figure 8.11 Redis Concurrency Test 50

Figure 8.12 RabbitMQ Queue Processing 51

Figure D.1 Additional screenshot 1 56

Figure D.2 Additional screenshot 2 56

LIST OF TABLES

Table 0.1 List of Abbreviations 12

Table 1.1 Limitations of existing ticket booking approaches 14

Table 1.2 Scope of the project 16

Table 1.3 Project team and roles 17

Table 2.1 Summary of related areas and their use in the project 20

Table 3.1 Problem analysis: situations in the booking interval 21

Table 3.2 Functional requirements 21

Table 3.3 Non-functional requirements 22

Table 3.4 Hardware requirements 22

Table 3.5 Software requirements 22

Table 4.1 Architecture components and their roles 25

Table 4.2 Principal database entities 33

Table 5.1 API operations of the system 34

Table 6.1 Testing strategy 40

Table 6.2 Test cases for the Ticket Booking System 42

Table 7.1 Summary of results by module 44

Table 9.1 API endpoints 55

Table 9.2 Users table structure 55

Table 9.3 Shows table structure 55

Table 9.4 Seats table structure 56

Table 9.5 Bookings table structure 56

Table 9.6 Test execution record 57

LIST OF ABBREVIATIONS

Table 0.1 List of Abbreviations

Abbreviation	Full Form
API	Application Programming Interface
B.Tech	Bachelor of Technology
CSE	Computer Science and Engineering
DFD	Data Flow Diagram
ER / ERD	Entity Relationship (Diagram)
HTTP	Hypertext Transfer Protocol
ID	Identifier
MySQL	A relational database management system that uses Structured Query Language
OS	Operating System
REST	Representational State Transfer
SQL	Structured Query Language
TTL	Time To Live (the temporary expiry period of a Redis key)
UI	User Interface
UML	Unified Modeling Language
VS Code	Visual Studio Code

CHAPTER 1 – INTRODUCTION

1.1 Introduction

Entertainment and event venues such as cinemas, theatres and concert halls depend on an organised method of selling a limited number of seats. In the past this was done at a physical counter, where one person served one customer at a time and the seating chart was kept on paper or in a simple local system. Today the same activity is performed through web applications, in which thousands of users can view the same show, look at the same seating layout and press the "book" button within the same few seconds.

The *Ticket Booking System* developed in this project is a web-based application for booking tickets to movies, events and shows. A user can register and log in, browse the available shows, select a date and time, view real-time seat availability, select seats, make payment, receive a booking ID and digital ticket, view booking history, and cancel a booking. Although these features are familiar, the main technical interest of the project lies in how the system behaves when several users compete for the same seat. The project therefore combines three technologies in its backend: Redis for distributed seat locking, RabbitMQ for message queuing, and MySQL for persistent storage.

1.2 Background

The movement of ticket sales from physical counters to web applications has changed the way in which seat inventory must be managed. In a counter-based system the sale of each ticket is naturally serialised, because only one transaction is handled at a time. In a web-based system, many requests reach the server in parallel, and a seat that appears free to several users at the same moment may be requested by all of them before any one request has completed. This situation is known as a race condition.

Relational databases provide transactions and constraints that help to maintain correctness, but a database-only approach forces every competing request to reach the database and wait for it, and it does not by itself provide a convenient way of holding a seat temporarily while a user completes payment. For this reason, ticket booking applications commonly place a short-lived reservation in front of the database. A distributed lock held in an in-memory data store such as Redis serves this purpose in the present project. **[REFERENCE REQUIRED]**

Booking systems also perform work that does not need to finish within the user's request, such as payment processing and refund handling. A message queue allows the web application to accept such a request quickly and let a separate consumer process it. RabbitMQ is used in this project for this purpose. The project is therefore best understood as a study of database systems engineering and distributed backend development: it addresses data consistency, shared locking and asynchronous processing in the setting of a realistic application.

[REFERENCE REQUIRED]

1.3 Problem Statement

The major technical problem addressed by this project is Smart Leave Management System, in which multiple users may try to book the same seat at the same time. When the availability of a seat is checked and its booking is recorded as two separate steps, two requests can both pass the check before either has recorded its booking. The result is a double booking: the same seat is sold to more than one user.

A second part of the problem concerns the handling of the booking lifecycle. A booking involves seat selection, payment, confirmation and, in some cases, cancellation with a refund. If all of these operations are executed directly inside the request-response cycle, the system becomes slow and fragile under load. The problem to be solved is therefore to design and build a ticket booking system in which (i) a seat cannot be successfully booked by two users, (ii) a seat selected by one user is held temporarily while that user completes the booking, and (iii) booking, payment and refund requests are processed in an efficient and orderly manner.

1.4 Existing System

In a traditional existing system, tickets are sold at a physical counter and the availability of seats is maintained manually or in a local application. Users must visit the venue, choose from the seats that the operator reports as available, and pay at the counter. Basic online booking systems reduce the need to travel, but a simple implementation typically reads the seat status from a database, shows it to the user, and updates the status when the user completes the booking, without any mechanism that protects a seat during the interval between selection and confirmation.

The description above is a general characterisation of existing approaches. Any specific existing system described in the project presentation **[TO BE PROVIDED]** should be added here, together with a reference where available. **[REFERENCE REQUIRED]**

1.5 Limitations of Existing System

The existing approaches described in Section 1.4 have the following limitations in relation to the problem addressed by this project.

Table 1.1 Limitations of existing ticket booking approaches

Limitation	Explanation
Manual counter booking	Requires physical presence, serves one customer at a time and gives users no way to view availability remotely.
No real-time seat availability	Users may select seats that were already taken by another user, because the information shown is out of date.

Limitation	Explanation
Risk of double booking	Without protection between selection and confirmation, two simultaneous requests can both succeed for the same seat.
No temporary seat hold	A seat chosen by one user is not reserved while that user completes payment, so another user may take it.
Synchronous processing	Payment and refund processing performed inside the user's request slows the response and makes the system fragile under load.
Heavy dependence on the database	All competing requests must be resolved at the database, increasing contention on the same records.

1.6 Proposed System

The proposed *Ticket Booking System* is a web-based application with a React.js frontend, a Java and Spring Boot backend, and a MySQL database. Two further components are added to manage concurrency and workload. Redis is used for distributed seat locking: when a user selects a seat, the backend attempts to place a lock on that seat, and the lock has a temporary expiry. If another user attempts to select the same seat while the lock exists, that second request receives a conflict. RabbitMQ is used to queue booking, payment and refund-related requests, so that they are processed efficiently rather than inside the user's request.

The booking process follows a fixed sequence: the user selects a show and seats; Redis attempts to lock the selected seat; the booking is processed through RabbitMQ; the booking is stored or updated in MySQL; and, after successful confirmation, the seat becomes BOOKED and the Redis lock is released. Cancellation of a booking initiates refund processing.

1.7 Advantages of Proposed System

- **Prevention of double booking:** the use of a Redis seat lock ensures that the same seat is not successfully booked by two users at the same time.
- **Temporary seat holding:** a selected seat is held for a limited period through the lock's expiry, so that a user can complete the booking, and the seat is not held indefinitely.
- **Real-time seat availability:** users can view the availability of seats before choosing them.
- **Orderly request processing:** the queuing of booking, payment and refund requests through RabbitMQ allows these requests to be processed in an organised manner.
- **Reliable persistent storage:** MySQL stores users, shows, seats, bookings and their status, giving a durable record of every booking.
- **Complete user journey:** registration, browsing, seat selection, payment, digital ticket, booking history and cancellation are available in one application.
- **Separation of concerns:** the frontend, backend, database, lock layer and queue layer are separate components that can be understood and modified independently.

These advantages are stated qualitatively. The report does not claim that the system is completely secure or unlimitedly scalable, and no measured performance values are given unless they are supplied by the project team. **[TO BE PROVIDED]**

1.8 Objectives

The objectives of the project are as follows:

1. To design and develop a web-based ticket booking system for movies, events and shows.
2. To provide user registration and login, show browsing, date and time selection, real-time seat availability and seat selection.
3. To prevent two users from booking the same seat simultaneously by using Redis for distributed seat locking with a temporary expiry.
4. To use RabbitMQ to queue booking, payment and refund-related requests for efficient processing.
5. To store users, shows, seats, bookings and related status information in a MySQL database.
6. To provide a booking ID and digital ticket after a successful booking, a booking history for each user, and a cancellation facility that initiates refund processing.
7. To test the system, including a dedicated test of two users attempting to book the same seat at the same time.

1.9 Scope of the Project

The scope of the project is a web application for ticket booking that is used through a browser (Chrome or Edge) on Windows or Linux. The scope covers the functions listed in Section 1.1, the concurrency control and queue-based processing described above, and the storage of the related data in MySQL. Features that are not part of the present implementation are listed as future scope and are not claimed as implemented.

Table 1.2 Scope of the project

Within the scope (this project)	Outside the scope (future scope)
Web-based application using React.js	Android / iOS mobile application
Registration and login; show browsing; date and time selection	Cloud deployment
Real-time seat availability and seat selection	Real payment gateway integration
Redis distributed seat locking with temporary expiry	Email / SMS notifications
RabbitMQ queuing of booking, payment and refund requests	Analytics and AI recommendations
MySQL storage of users, shows, seats, bookings and status	Support for multiple theatres

Within the scope (this project)	Outside the scope (future scope)
Booking ID, digital ticket, booking history, cancellation and refund processing	Stronger security measures

Whether payment in the present implementation is simulated or connected to an external service is **[TO BE PROVIDED]**; the inclusion of a real payment gateway in the future scope indicates that the present payment function does not use one.

1.10 Organization of the Report

The report is organised into nine chapters. Chapter 1 introduces the project, its background, problem statement, existing and proposed systems, objectives and scope. Chapter 2 presents a general survey of related work on ticket booking, concurrent booking, distributed locking, message queuing and database management. Chapter 3 analyses the system requirements and the feasibility of the project. Chapter 4 describes the system design, including the architecture, modules and diagrams. Chapter 5 explains the implementation of each part of the system and the booking flow. Chapter 6 describes the testing strategy and test cases. Chapter 7 presents and discusses the results. Chapter 8 contains placeholders for the application screenshots. Chapter 9 concludes the report and discusses the limitations and future scope. The report ends with references and appendices.

Project Team and Responsibilities

Table 1.3 Project team and roles

Name	Roll Number	Role
Rayanki HimaVarshini	2520030141	Team Lead and Backend/Database Developer
Gurrapu Nikshitha	2520030512	Frontend Developer
Vemua Naga Venkata Sai Chandu	2520030522	Concurrency and Integration Developer

CHAPTER 2 – LITERATURE SURVEY / RELATED WORK

Note: No research papers or citations were supplied as part of the project material. This chapter is therefore written as a general related-work discussion. Places where a published source should be cited are marked **[REFERENCE REQUIRED]**, and no specific paper has been invented.

2.1 Introduction

The literature relevant to this project lies in five areas: approaches to ticket booking, online ticket booking systems, the problem of concurrent booking, distributed locking, and message queuing, together with the management of booking data in a database. This chapter discusses each area in general terms and explains how it relates to the design of the *Ticket Booking System*.

2.2 Existing Ticket Booking Approaches

Ticket booking has been carried out through several approaches. The earliest is the counter-based approach, in which an operator sells tickets at the venue and records the sale manually or in a local system. Telephone booking extends this approach by allowing an operator to reserve seats for a caller, but it still depends on a human operator. Web-based booking removes the operator and lets users reserve seats directly. Each approach differs in how it controls access to a limited set of seats: the counter approach relies on the operator, while the web-based approach must rely on software mechanisms. **[REFERENCE REQUIRED]**

2.3 Online Ticket Booking Systems

Online ticket booking systems generally provide a catalogue of shows, a seat map, a payment step and a confirmation. The usual architecture separates a browser-based client from a server-side application that owns the business rules and a database that stores the records. Frameworks such as React.js for the client and Spring Boot for the server are commonly used to build such applications, with REST interfaces connecting the two. A well-designed system also gives the user feedback at every step, for example by indicating that a seat is no longer available. The present project follows this general architecture and adds a specific mechanism for handling concurrent requests. **[REFERENCE REQUIRED]**

2.4 Concurrent Booking Problems

Concurrent booking problems arise when several requests act on the same data at the same time. The classical example is the “check-then-act” race condition: a request reads that a seat is free, and acts on that belief by booking it, but a second request performs the same read before the first has written its result. Both requests then believe that the seat is free and both book it. The consequences are double booking, inconsistent records, and a poor user experience. **[REFERENCE REQUIRED]**

Solutions in the general literature fall into several groups: database-level mechanisms such as transactions and row locking, unique constraints that reject a second booking of the same seat, application-level locks held within a single server process, and distributed locks shared among several server instances. An application-level lock is not sufficient when the backend can run as more than one instance, because each instance has its own memory; this motivates a lock held in a shared external store. **[REFERENCE REQUIRED]**

2.5 Distributed Locking

A distributed lock is a mutual-exclusion mechanism whose state is held in a store that is visible to all participants, so that only one participant can hold the lock for a given resource at a given time. An in-memory key-value store such as Redis is commonly used for this purpose because it offers fast operations and supports keys that expire automatically after a fixed period. The expiry is important: if a lock holder fails or abandons its work, the lock is eventually released and the resource does not remain blocked forever. In the present project, Redis holds a temporary lock on a seat while a user is completing a booking. **[REFERENCE REQUIRED]**

A lock with an expiry also introduces a design question: the period must be long enough for the user to finish the booking but short enough that an abandoned seat is returned to other users. The expiry period used in the project is **[TO BE PROVIDED]**.

2.6 Message Queuing

A message queue decouples the component that produces a request from the component that processes it. The producer places a message on the queue and continues; one or more consumers take messages from the queue and process them. This approach smooths the effect of bursts of requests, allows slow operations to be performed outside the user’s request, and keeps requests in an orderly sequence. RabbitMQ is a message broker used for this style of processing. In the present project, booking, payment and refund-related requests are queued through RabbitMQ. **[REFERENCE REQUIRED]**

2.7 Database Management in Booking Systems

A booking system must store its data durably and consistently. Relational database management systems such as MySQL organise data into tables with defined relationships and support transactions, so that a group of related changes either all take effect or none does. For a booking system the principal records are users, shows, seats and bookings, together with status information that indicates, for example, whether a seat is BOOKED. The database is the permanent source of truth, while temporary structures such as the Redis lock only govern access during the booking interval. **[REFERENCE REQUIRED]**

2.8 Summary of Literature Survey

The general related work shows that ticket booking has moved from counter-based to web-based operation, that web-based booking introduces the concurrent booking problem, and that a combination of a durable relational database, a shared lock with expiry and an asynchronous message queue is a reasonable way to address it. The following table summarises how each area is applied in this project.

Table 2.1 Summary of related areas and their use in the project

Area	General idea	Use in this project
Ticket booking approaches	Counter, telephone and web-based booking	Web-based booking with React.js and Spring Boot
Concurrent booking	Race conditions cause double booking	Dedicated concurrency test: two users, same seat
Distributed locking	Shared lock with automatic expiry	Redis temporary seat lock
Message queuing	Decouples producers from consumers	RabbitMQ queues for booking, payment and refund requests
Database management	Durable, consistent storage	MySQL for users, shows, seats, bookings and status

CHAPTER 3 – SYSTEM ANALYSIS

3.1 Problem Analysis

The analysis of the problem begins with the sequence of events in a booking. A user selects a seat; the system must confirm that the seat is still available, hold it for the user, accept payment, record the booking, and show a ticket. Between the selection and the confirmation there is an interval during which the seat is neither free nor sold. If this interval is not handled, other users can select the same seat, and the system faces the following situations.

Table 3.1 Problem analysis: situations in the booking interval

Situation	Risk	Handling in the proposed system
Two users select the same seat at the same time	Double booking	Redis lock granted to the first valid request; the second request receives a conflict
A user selects a seat but does not complete the booking	Seat blocked indefinitely	The Redis lock has a temporary expiry
Many booking, payment and refund requests arrive together	Slow or unreliable responses	Requests are queued through RabbitMQ
A booking is confirmed	Seat must not be sold again	Seat becomes BOOKED in MySQL and the Redis lock is released
A booking is cancelled	Refund must be handled	Cancellation initiates refund processing

3.2 Functional Requirements

Table 3.2 Functional requirements

ID	Requirement
FR-01	The system shall allow a user to register and log in.
FR-02	The system shall allow a user to browse shows (movies, events and shows).
FR-03	The system shall allow a user to select a date and time for a show.
FR-04	The system shall display real-time seat availability.
FR-05	The system shall allow a user to select seats.
FR-06	The system shall place a temporary Redis lock on a selected seat and return a conflict to another user attempting the same seat.
FR-07	The system shall accept payment for a booking.
FR-08	The system shall process booking, payment and refund-related requests through RabbitMQ.
FR-09	The system shall store and update bookings in MySQL, mark the seat BOOKED after confirmation and release the Redis lock.
FR-10	The system shall provide a booking ID and digital ticket after a successful booking.
FR-11	The system shall allow a user to view booking history.
FR-12	The system shall allow a user to cancel a booking, which initiates refund processing.
FR-13	Administrator operations: **[TO BE PROVIDED]**

3.3 Non-Functional Requirements

Table 3.3 Non-functional requirements

Requirement	Description
Consistency	A seat must not be successfully booked by two users.
Responsiveness	Seat availability and booking status should be shown promptly. Numerical response-time targets: **[TO BE PROVIDED]**.
Reliability	Booking data must be stored durably in MySQL; seat locks must expire so that seats are not blocked indefinitely.
Usability	The interface must allow a user to complete the booking journey with clear steps.
Maintainability	The frontend, backend, database, lock layer and queue layer are kept as separate components.
Compatibility	The application must work in the Chrome and Edge browsers on Windows or Linux.
Security	Security measures implemented in the present version: **[TO BE PROVIDED]**. Stronger security is listed as future scope.

3.4 Hardware Requirements

Table 3.4 Hardware requirements

Component	Requirement
Processor	**[TO BE PROVIDED]**
RAM	**[TO BE PROVIDED]**
Hard disk / storage	**[TO BE PROVIDED]**
Display	**[TO BE PROVIDED]**
Network	**[TO BE PROVIDED]**

The hardware used for development and testing was not specified in the project material and must be supplied by the team.

3.5 Software Requirements

Table 3.5 Software requirements

Category	Software	Version
Frontend	React.js	**[TO BE PROVIDED]**
Backend	Java with Spring Boot	**[TO BE PROVIDED]**
Database	MySQL	**[TO BE PROVIDED]**
Distributed locking	Redis	**[TO BE PROVIDED]**
Message queue	RabbitMQ	**[TO BE PROVIDED]**
Build tool	Maven	**[TO BE PROVIDED]**
API testing	Postman	**[TO BE PROVIDED]**
Development environment	Visual Studio Code	**[TO BE PROVIDED]**
Operating system	Windows / Linux	**[TO BE PROVIDED]**

Category	Software	Version
Browser	Google Chrome / Microsoft Edge	**[TO BE PROVIDED]**

3.6 User Requirements

Two kinds of users are considered. A **registered user** needs to register and log in, browse shows, choose a date and time, see which seats are available, select seats without being affected by another user taking the same seat, pay, obtain a ticket, review past bookings and cancel a booking when required. An **administrator** needs facilities to manage the data on which the booking process depends; the specific administrator operations are **[TO BE PROVIDED]**.

3.7 System Constraints

- A seat lock is temporary; its expiry duration is **[TO BE PROVIDED]**, and a user must complete the booking within that period.
- The system requires Redis, RabbitMQ and MySQL to be running together with the Spring Boot backend.
- Payment is handled as described in Section 5.12; a real payment gateway is listed under future scope.
- Support for multiple theatres is future scope, so the present system is limited accordingly.
- The application is intended for the Chrome and Edge browsers.
- No measured performance or load figures are included unless supplied by the team.

3.8 Feasibility Analysis

3.8.1 Technical Feasibility

The project uses established technologies: React.js, Java with Spring Boot, MySQL, Redis and RabbitMQ, built with Maven, tested with Postman and developed in Visual Studio Code. Each technology performs a clearly defined role, and they can be run together on a Windows or Linux machine. The team members have divided the work among backend and database development, frontend development, and concurrency and integration. The project is therefore technically feasible.

3.8.2 Economic Feasibility

The project uses only the tools listed in the technology stack and does not use a paid payment gateway, cloud hosting or notification service, which are listed as future scope. The development cost is therefore mainly the time of the team and the use of computers already available to them. The exact cost, including any licence or hosting cost, is **[TO BE PROVIDED]**.

3.8.3 Operational Feasibility

The workflow of the system follows the familiar steps of online ticket booking, so users can operate it without special training. Real-time seat availability and the clear conflict response when a seat is already taken help users understand what has happened. The booking history and cancellation functions allow users to manage their bookings. The system is therefore operationally feasible.

CHAPTER 4 – SYSTEM DESIGN

4.1 System Architecture

The *Ticket Booking System* follows a layered, distributed architecture. A user interacts with a React.js frontend through a browser. The frontend communicates with a Java and Spring Boot backend through REST APIs. The backend coordinates three supporting components: Redis for distributed seat locking, RabbitMQ for queuing booking, payment and refund-related requests, and MySQL for persistent storage. Figure 4.1 shows the high-level architecture.

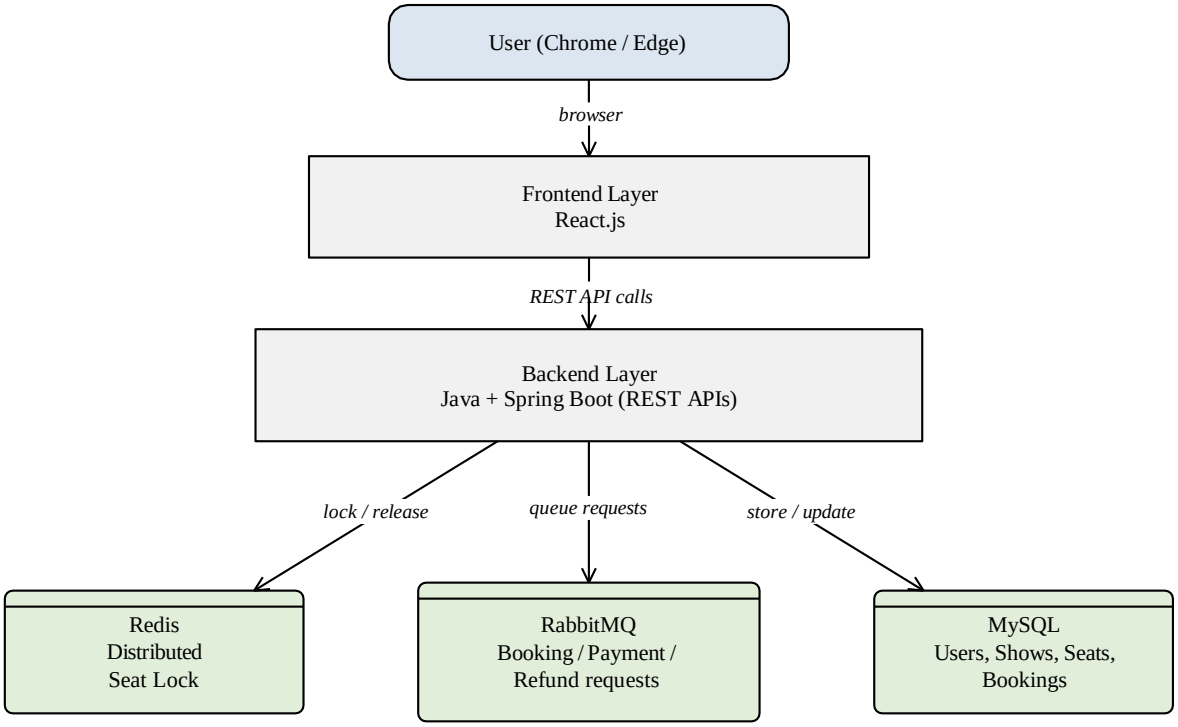


Figure 4.1 High-level architecture of the Ticket Booking System

4.2 Architecture Components

Table 4.1 Architecture components and their roles

Component	Technology	Role in the system
Frontend	React.js	Presents pages to the user: login, registration, shows, seat selection, booking, payment, ticket and bookings.
Backend	Java, Spring Boot	Implements the REST APIs and coordinates locking, queuing and database updates.
Database	MySQL	Stores users, shows, seats, bookings and related status information.
Distributed lock	Redis	Holds a temporary lock on a seat so that two users cannot book it simultaneously.
Message queue	RabbitMQ	Queues booking, payment and refund-related requests for efficient processing.
Build tool	Maven	Builds the Java backend project and manages its dependencies.

Component	Technology	Role in the system
API testing	Postman	Used to send requests to the backend APIs and examine the responses.

4.3 Frontend Layer

The frontend layer is developed with React.js and is the only part of the system that the user sees. It contains the pages needed for the booking journey: login and registration, the list of shows, selection of date and time, the seat selection page, the booking and payment pages, the digital ticket, the user's bookings and, for administrators, a dashboard. The frontend does not decide whether a seat can be booked; it sends the user's actions to the backend and displays the response, including a conflict message if another user has already locked the seat. The detailed component structure of the frontend is **[TO BE PROVIDED]**.

4.4 Backend Layer

The backend layer is developed in Java with Spring Boot and built with Maven. It exposes REST APIs to the frontend and contains the business logic of the system. For each request it validates the operation, communicates with Redis to obtain or release a seat lock, places booking, payment and refund-related requests on RabbitMQ, and reads or updates the data in MySQL. The backend is therefore the coordinator that keeps the three supporting components consistent with one another. The package structure of the backend is **[TO BE PROVIDED]**.

4.5 Database Layer

The database layer uses MySQL, a relational database management system. It stores users, shows, seats and bookings, together with related status information such as whether a seat is BOOKED. MySQL is the permanent record of the system: a booking is regarded as complete only after it has been stored or updated here. The conceptual design of the database is given in Sections 4.18 and 4.19.

4.6 Redis Distributed Locking Layer

Redis provides the distributed locking layer. When a user selects a seat, the backend attempts to place a lock on that seat in Redis. The lock has a temporary expiry, so that a seat selected by a user who does not complete the booking becomes available again without manual intervention. While the lock exists, any other request for the same seat receives a conflict. After the booking is successfully confirmed and the seat becomes BOOKED in MySQL, the Redis lock is released. The structure of the lock key and the expiry duration are **[TO BE PROVIDED]**.

4.7 RabbitMQ Message Queue Layer

RabbitMQ provides the message queue layer. Booking, payment and refund-related requests are placed on queues by the backend and are processed in an orderly manner, instead of being executed entirely within the user's request. This layer allows the system to accept requests promptly and to process them efficiently. The number and names of the queues and the structure of the messages are **[TO BE PROVIDED]**.

4.8 User Module

The user module manages registration and login. A new user registers and later logs in to access the booking functions. The module also gives each user access to their own booking history. The fields stored for a user and the authentication mechanism used are **[TO BE PROVIDED]**.

4.9 Show and Seat Module

The show and seat module manages the information that a user sees before booking. It supplies the list of shows (movies, events and shows), the available dates and times, and the seats of the selected show together with their real-time availability. It also records the status of each seat, which becomes BOOKED when a booking is confirmed.

4.10 Booking and Payment Module

The booking and payment module handles the creation of a booking for the selected seats, the processing of payment, the confirmation of the booking, the generation of a booking ID and digital ticket, and the cancellation of a booking with refund processing. These operations are carried out through RabbitMQ and recorded in MySQL.

4.11 Concurrency and Queue Module

The concurrency and queue module integrates Redis and RabbitMQ with the backend. It is responsible for acquiring and releasing the seat lock, for returning a conflict when a seat is already locked, and for publishing and consuming the booking, payment and refund requests. This module is the core of the project's response to the concurrent booking problem and was the responsibility of the Concurrency and Integration Developer.

4.12 Admin Module

The project material includes an administrator interface (Figure 8.10, Admin Dashboard). The specific operations available to an administrator, such as the management of shows, seats or bookings, are **[TO BE PROVIDED]**. No administrator feature is claimed in this report until it is confirmed by the project team.

4.13 Data Flow Diagram

Figure 4.2 shows the data flow of the system at the first level of detail. The user interacts with five processes: registration and login, browsing of shows and seat status, seat selection with lock acquisition, booking and payment, and cancellation and refund. The processes use three data stores: the MySQL database, the Redis seat locks and the RabbitMQ queues.

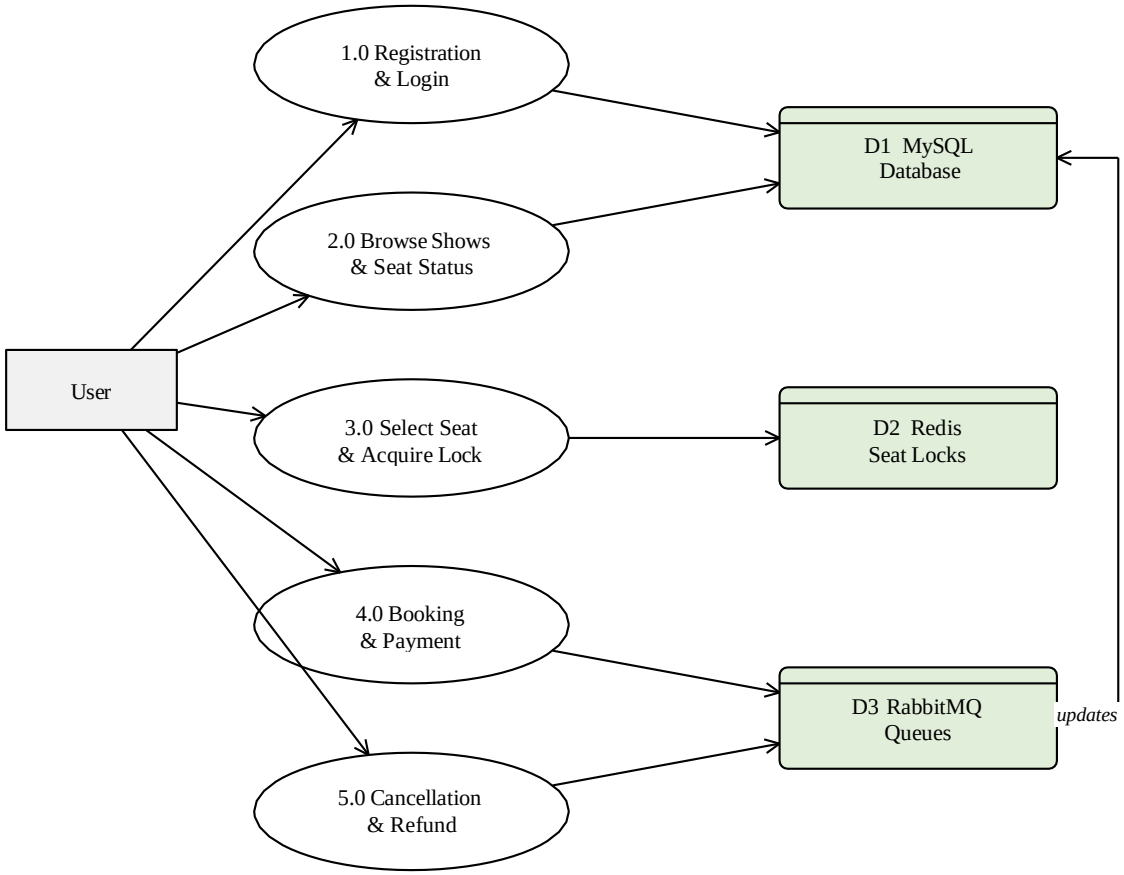


Figure 4.2 Level-1 Data Flow Diagram

4.14 Use Case Diagram

Figure 4.3 shows the use cases of the system. The registered user performs the booking-related use cases. The administrator is shown with a single use case whose content is **[TO BE PROVIDED]**.

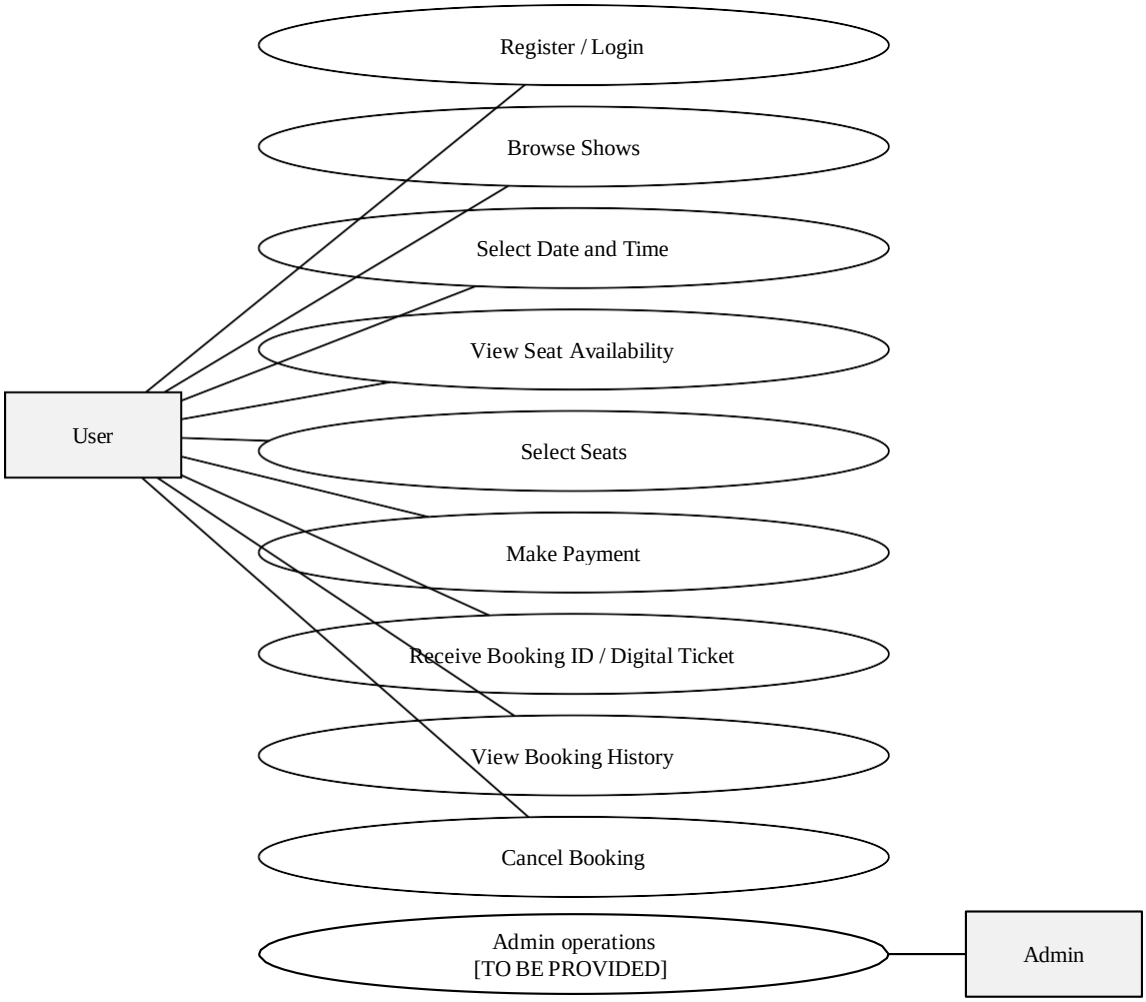


Figure 4.3 Use case diagram of the Ticket Booking System

4.15 Activity Diagram

Figure 4.4 shows the activity flow of a seat booking, including the point at which a second user receives a conflict.

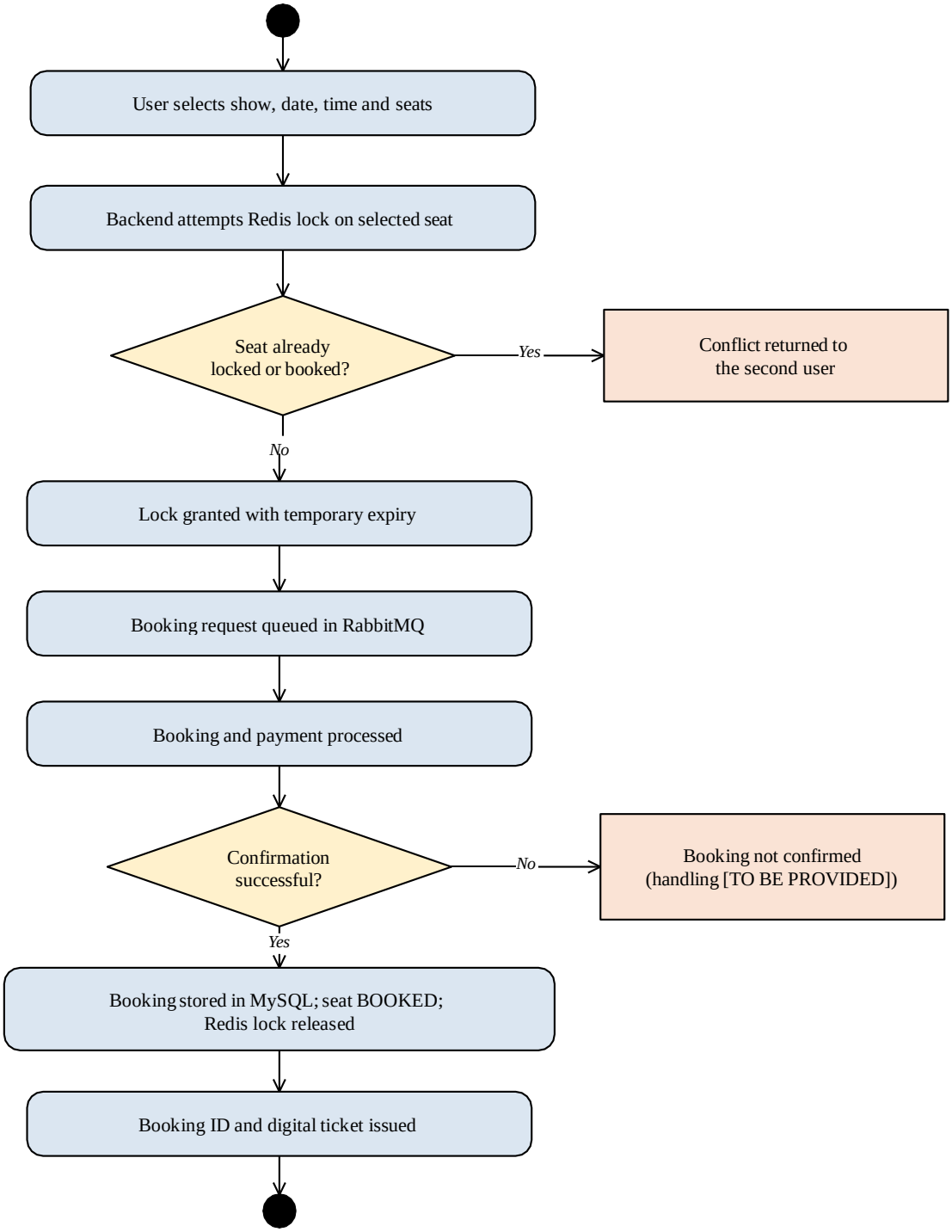


Figure 4.4 Activity diagram of the seat booking process

4.16 Sequence Diagram

Figure 4.5 shows the order of interactions between the user, the frontend, the backend, Redis, RabbitMQ and MySQL during a successful booking. The numbers correspond to the steps of the booking flow described in Section 5.11.

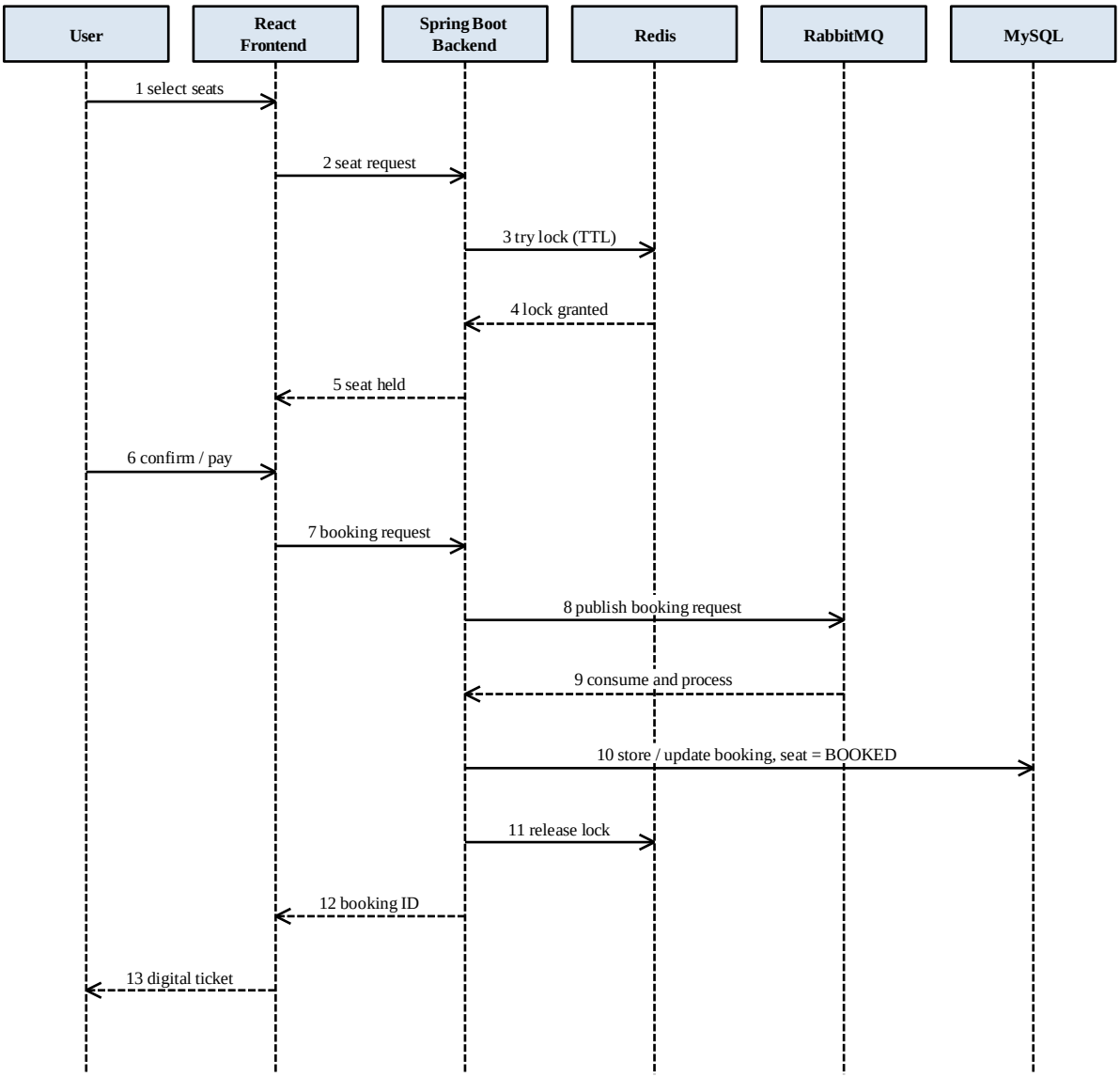


Figure 4.5 Sequence diagram of a successful booking

4.17 Class Diagram

Figure 4.6 shows a conceptual class diagram based on the four principal entities named in the project material. The attributes and methods of each class are **[TO BE PROVIDED]**, because the field-level design was not included in the material supplied.

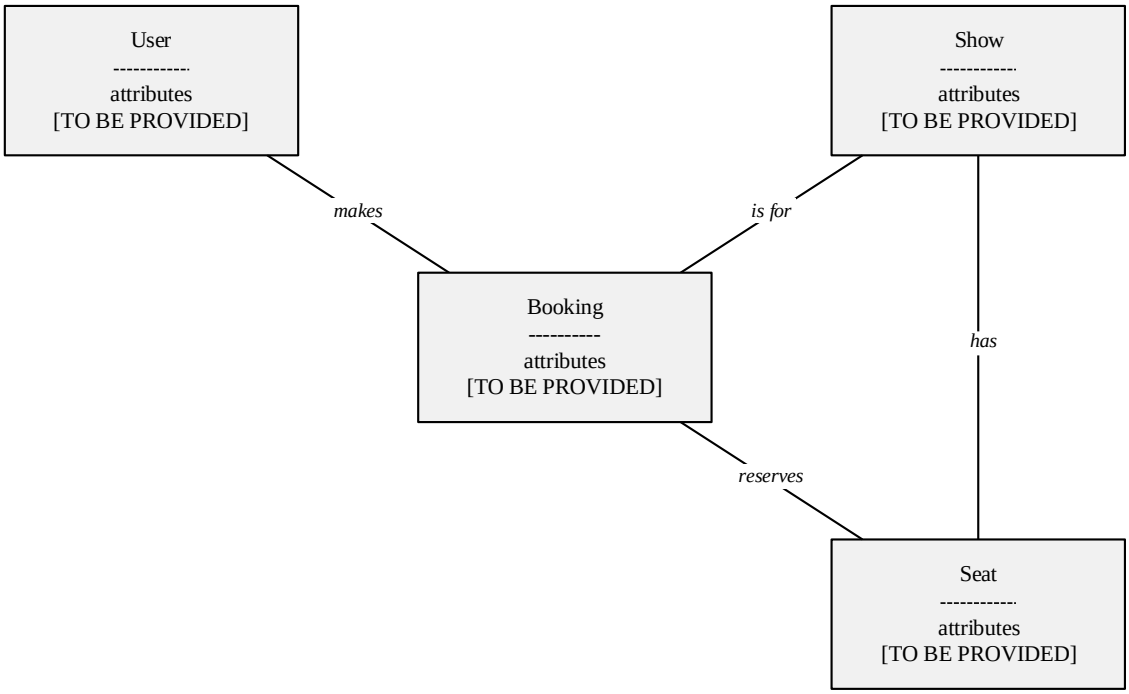


Figure 4.6 Conceptual class diagram

4.18 Entity Relationship Diagram

Figure 4.7 presents the conceptual entity relationship diagram. A user makes bookings; each booking is for a show and reserves seats; each show has seats. The cardinalities shown are the natural reading of the booking process and are to be confirmed against the final schema **[TO BE PROVIDED]**.

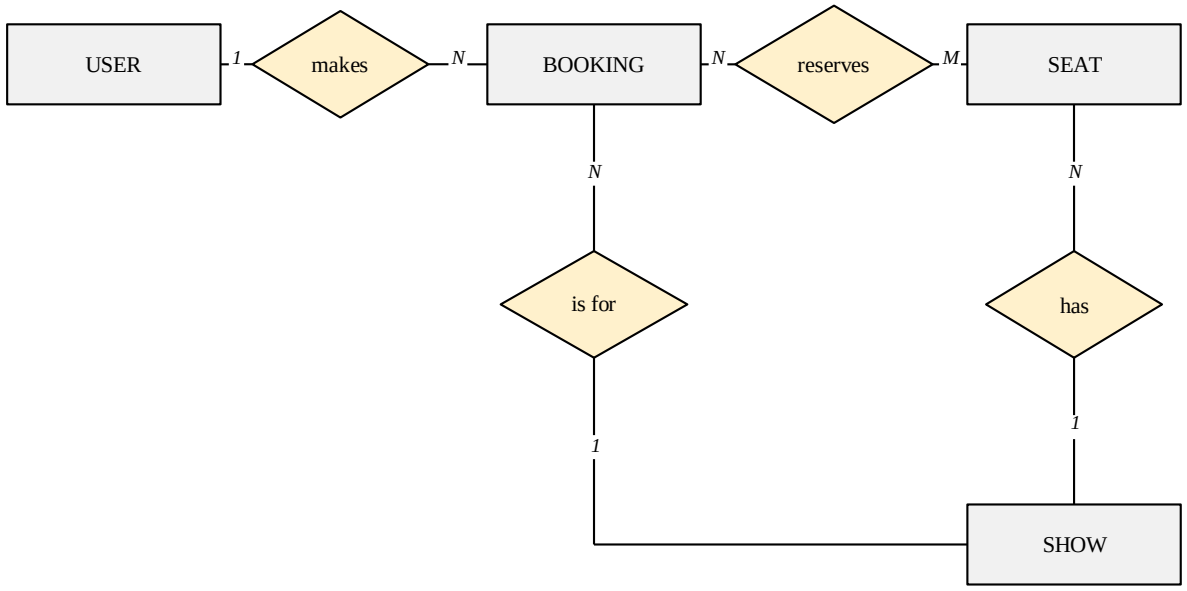


Figure 4.7 Conceptual entity relationship diagram

4.19 Database Design

The MySQL database stores users, shows, seats, bookings and related status information. The following table lists the principal entities and their purpose. The actual table names, columns, data types, keys and constraints were not part of the supplied material and are marked **[TO BE PROVIDED]**; they are to be completed from the database schema of the project and entered in Appendix C.

Table 4.2 Principal database entities

Entity	Purpose	Columns / keys
Users	Stores registered users who log in and make bookings	**[TO BE PROVIDED]**
Shows	Stores movies, events and shows with their date and time	**[TO BE PROVIDED]**
Seats	Stores the seats of a show and their status, including BOOKED	**[TO BE PROVIDED]**
Bookings	Stores each booking, its status and its relation to a user, show and seats	**[TO BE PROVIDED]**
Payment / refund status	Stores related status information for payments and refunds (separate table or fields to be confirmed)	**[TO BE PROVIDED]**

CHAPTER 5 – IMPLEMENTATION

5.1 Implementation Overview

The system was implemented as a React.js frontend and a Java and Spring Boot backend, using MySQL for storage, Redis for seat locking and RabbitMQ for message queuing. The backend project is built with Maven, the code was written in Visual Studio Code, and the APIs were exercised with Postman. The project was divided among the team members: backend and database development (Rayanki HimaVarshini), frontend development (Gurrapu Nikshitha), and concurrency and integration (Vemua Naga Venkata Sai Chandu). This chapter explains each part of the implementation. Source code excerpts are to be placed in Appendix A **[TO BE PROVIDED]**.

5.2 Frontend Implementation using React

The frontend is implemented with React.js and provides the pages listed in Section 4.3. It sends the user's actions to the backend and renders the responses. Real-time seat availability is displayed on the seat selection page, and a conflict response from the backend is shown to the user when the selected seat is already locked. The names of the components, the state management approach and the way in which seat availability is refreshed are **[TO BE PROVIDED]**.

5.3 Backend Implementation using Java and Spring Boot

The backend is implemented in Java using Spring Boot and built with Maven. It receives requests from the frontend through REST APIs, applies the booking rules, communicates with Redis and RabbitMQ, and reads and updates MySQL. It is the single component that coordinates the lock, the queue and the database in the booking flow. The classes, layers and configuration of the backend are **[TO BE PROVIDED]**.

5.4 REST API Implementation

The frontend communicates with the backend through REST APIs. The operations that the APIs must provide follow directly from the functions of the system and are listed below. The HTTP methods, URL paths and request and response formats are **[TO BE PROVIDED]** and are to be entered in Appendix B from the Postman collection used for testing.

Table 5.1 API operations of the system

Operation	Method / URL	Purpose
Register user	**[TO BE PROVIDED]**	Create a new user account
Login	**[TO BE PROVIDED]**	Authenticate a user
List shows	**[TO BE PROVIDED]**	Return the shows available for browsing

Operation	Method / URL	Purpose
Get show timings / seat availability	**[TO BE PROVIDED]**	Return date, time and seat status of a show
Select / lock seat	**[TO BE PROVIDED]**	Attempt a Redis lock on a seat
Create booking and pay	**[TO BE PROVIDED]**	Submit booking and payment requests
Get booking / ticket	**[TO BE PROVIDED]**	Return booking ID and digital ticket
Booking history	**[TO BE PROVIDED]**	Return the user's bookings
Cancel booking	**[TO BE PROVIDED]**	Cancel a booking and initiate refund processing

5.5 MySQL Database Implementation

MySQL stores the persistent data of the system: users, shows, seats, bookings and related status information. A seat's status is updated to BOOKED when a booking is confirmed, and the booking is stored or updated in the database as part of the booking processing. The database is the final record of every booking; the Redis lock exists only to control access during the booking interval. The schema, relationships and constraints are **[TO BE PROVIDED]** (Appendix C).

5.6 Redis Seat Lock Implementation

When the user selects a seat, the backend attempts to obtain a lock on that seat in Redis. The lock is created with a temporary expiry. If the lock is obtained, the request proceeds into the booking process. If the lock already exists, the request is rejected with a conflict response. After a booking is confirmed, the lock is released. Because the lock is held in Redis, which is shared by all backend instances, the protection does not depend on the memory of a single server process. The key format and the expiry value are **[TO BE PROVIDED]**.

5.7 RabbitMQ Queue Implementation

RabbitMQ is used to queue booking, payment and refund-related requests. The backend publishes a message when such a request is made, and a consumer processes the message and carries out the corresponding database updates. Queuing allows the system to accept requests promptly and process them in order. The queue names, message formats and consumer configuration are **[TO BE PROVIDED]**.

Figure 5.1 shows the role of RabbitMQ between the backend and MySQL.

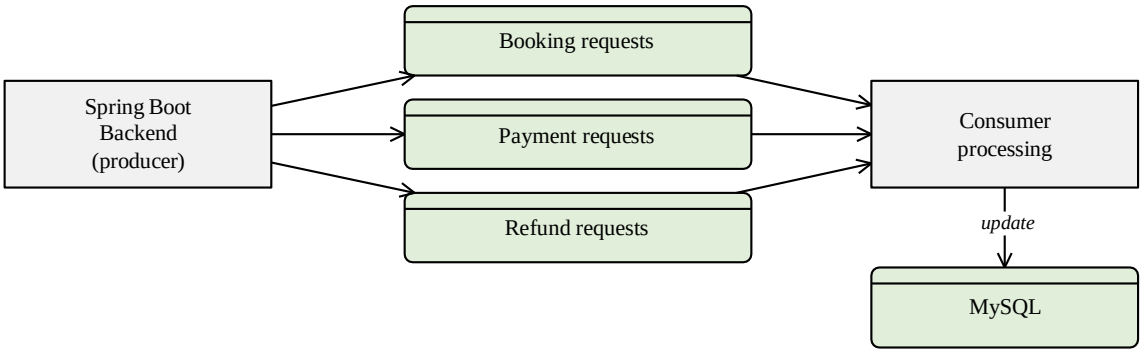


Figure 5.1 Queue processing through RabbitMQ (names of queues to be confirmed)

5.8 User Registration and Login

A new user registers with the system and then logs in. Only a logged-in user can proceed to book seats and view booking history. The registration fields, validation rules, password handling and session or token mechanism are **[TO BE PROVIDED]**. The corresponding screens are Figures 8.1 and 8.2.

5.9 Show Browsing

After logging in, the user can browse the shows. For a selected show the user chooses a date and time. The show data is read from MySQL and displayed by the frontend. The corresponding screen is Figure 8.3.

5.10 Seat Selection

For the selected show, date and time, the seat selection page displays the real-time availability of seats. The user selects the desired seats, and the selection triggers the Redis seat lock described in Section 5.6. If another user has already locked or booked a seat, a conflict is returned and the seat cannot be selected. The corresponding screen is Figure 8.4.

5.11 Booking Process

The booking process follows the sequence below. It is the core algorithm of the system and is illustrated in Figures 4.4 and 4.5.

1. The user selects a show and seats.
2. Redis attempts to lock the selected seat.
3. The lock has a temporary expiry.
4. If another user attempts the same seat, the second request receives a conflict.
5. The booking is processed through RabbitMQ.
6. The booking is stored or updated in MySQL.

7. After successful confirmation, the seat becomes BOOKED and the Redis lock is released.
8. If the booking is later cancelled, cancellation initiates refund processing.

The use of a temporary lock before the queue-based processing means that a seat is protected from the moment it is selected, while the queue ensures that the booking itself is processed in an orderly way. The corresponding screen is Figure 8.5.

5.12 Payment Process

After selecting seats, the user makes payment. Payment requests are handled through RabbitMQ. The project's future scope includes a real payment gateway; it is therefore expected that the present payment step is a simplified or simulated one. The exact payment method and its behaviour in the present implementation are **[TO BE PROVIDED]**. The corresponding screen is Figure 8.6.

5.13 Booking Confirmation

When the booking has been processed successfully, the booking is stored or updated in MySQL, the seat becomes BOOKED, and the Redis lock is released. The user receives a booking ID. The handling of a booking that cannot be confirmed (for example, because payment fails or the lock expires) is **[TO BE PROVIDED]**.

5.14 Digital Ticket Generation

After confirmation, the user receives a booking ID and a digital ticket, which can be viewed in the application. The content and layout of the digital ticket are **[TO BE PROVIDED]**; the corresponding screen is Figure 8.7.

5.15 Cancellation and Refund

A user can view booking history and cancel a booking. Cancellation initiates refund processing, and the refund-related request is handled through RabbitMQ. Whether a cancelled seat is returned to the available state, any cancellation rules (such as time limits) and the refund status values are **[TO BE PROVIDED]**. The corresponding screens are Figures 8.8 and 8.9.

5.16 Admin Operations

An administrator dashboard is part of the project material (Figure 8.10). The operations performed by an administrator are **[TO BE PROVIDED]**. They will be described here once confirmed.

5.17 Concurrency Handling

Concurrency handling is the principal technical contribution of the project. Its aim is that the same seat must not be successfully booked by two users. The system achieves this by using a Redis lock as the arbiter between competing requests. Of two simultaneous requests for the same seat, the first valid request obtains the lock; the second request finds that the lock exists and receives a conflict. Figure 5.2 illustrates this behaviour.

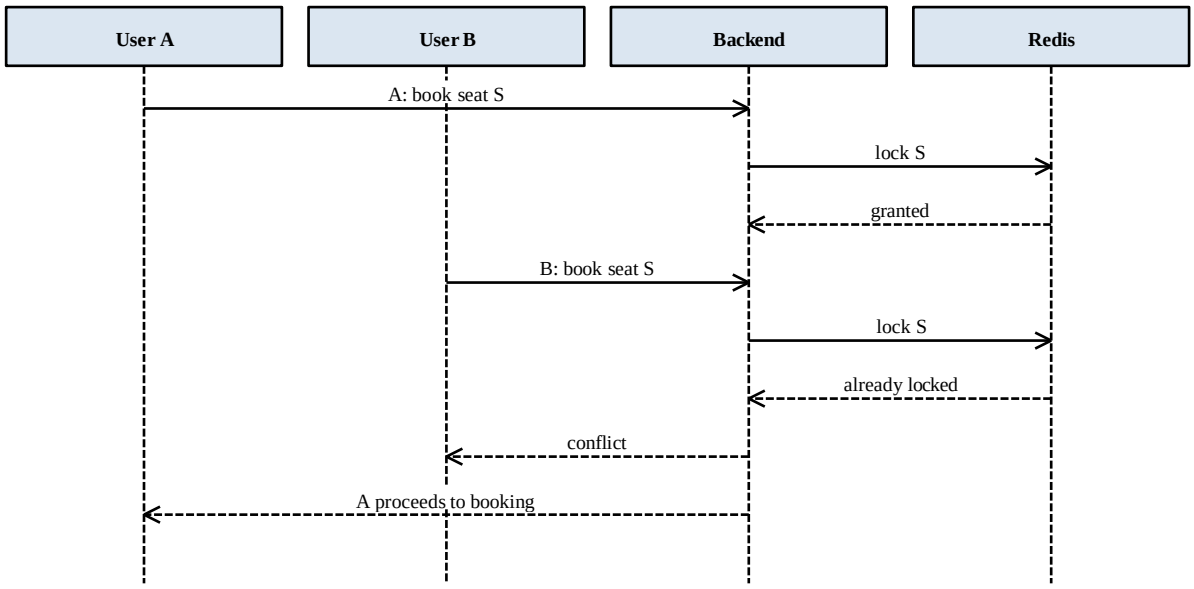


Figure 5.2 Two users attempting to book the same seat

The seat passes through the states shown in Figure 5.3. A seat is available, becomes locked temporarily when a user selects it, and becomes BOOKED after confirmation. If the lock expires without confirmation, the seat is available again. Because the lock has an expiry, a seat cannot remain blocked by a user who abandons the booking.

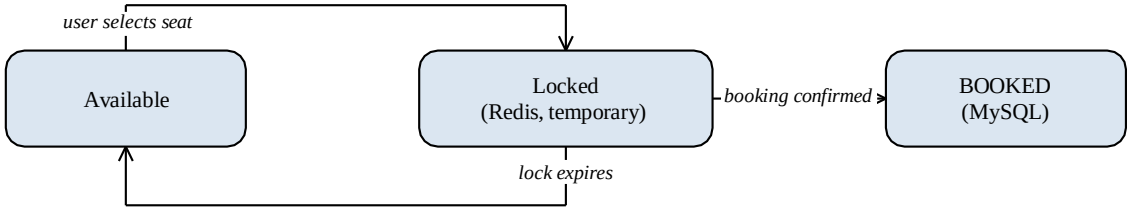


Figure 5.3 Seat states during the booking process

The state of a seat after cancellation is **[TO BE PROVIDED]**, because the project material states only that cancellation initiates refund processing. Redis and MySQL also have complementary roles: Redis protects the seat during the booking interval, and MySQL holds the permanent BOOKED status. Together with queue-based processing, this combination is the system's mechanism for avoiding double booking. The report does not claim that this

guarantee has been verified under heavy load; see Chapter 6 and Chapter 7.

CHAPTER 6 – TESTING

6.1 Testing Introduction

Testing verifies that the system behaves as specified and that its main technical claim holds: that the same seat is not successfully booked by two users. Because the system combines a frontend, a backend, a database, a lock layer and a queue layer, testing must examine each part separately and the parts together. This chapter describes the testing strategy and the test cases. **Actual results have not been supplied with the project material; wherever a result is required, it is marked [TO BE VERIFIED]** and must be filled in from the team's real test runs.

6.2 Testing Strategy

Table 6.1 Testing strategy

Level	Objective	Tool / method
Unit testing	Check individual backend and frontend units	**[TO BE PROVIDED]**
Integration testing	Check that frontend, backend, MySQL, Redis and RabbitMQ work together	Manual / **[TO BE PROVIDED]**
API testing	Check each REST API for valid and invalid requests	Postman
Functional testing	Check each user function from the browser	Chrome / Edge
Database testing	Check that data is stored and updated correctly in MySQL	MySQL queries **[TO BE PROVIDED]**
Redis lock testing	Check lock acquisition, conflict and expiry	**[TO BE PROVIDED]**
RabbitMQ testing	Check that requests are queued and consumed	**[TO BE PROVIDED]**
Concurrency testing	Check that two simultaneous requests for one seat cannot both succeed	**[TO BE PROVIDED]**

6.3 Unit Testing

Unit testing examines the smallest parts of the system in isolation, for example the logic that decides whether a seat lock can be granted, or a frontend component that displays seat status. The units tested, the framework used and the results are **[TO BE PROVIDED]**.

6.4 Integration Testing

Integration testing checks the combined behaviour of the components. The main integration path is: frontend request, backend, Redis lock, RabbitMQ message, consumer processing, MySQL update and response to the user. The scenarios tested and results are **[TO BE VERIFIED]**.

6.5 API Testing

The REST APIs were tested with Postman. For each API, valid requests should return the expected response and invalid requests should be rejected with an appropriate message. The Postman collection, the requests used and the screenshots of responses are **[TO BE PROVIDED]**; the results are **[TO BE VERIFIED]**.

6.6 Functional Testing

Functional testing follows the user's journey in the browser: registration, login, show browsing, date and time selection, seat selection, payment, receiving the digital ticket, viewing booking history and cancelling a booking. The test cases for this are listed in Section 6.11.

6.7 Database Testing

Database testing verifies that users, bookings and seat status are stored and updated correctly in MySQL. In particular, after a successful booking the booking must be recorded and the seat must have the status BOOKED. The queries used and the observations are **[TO BE PROVIDED]**.

6.8 Redis Lock Testing

Redis lock testing verifies three behaviours: a lock is granted for an available seat; a second request for the same locked seat receives a conflict; and a lock that is not confirmed expires so that the seat becomes available again. After a successful booking, the lock must be released. The expiry period tested is **[TO BE PROVIDED]**; the results are **[TO BE VERIFIED]**.

6.9 RabbitMQ Testing

RabbitMQ testing verifies that booking, payment and refund-related requests are placed on the queue and are consumed and processed, producing the expected database updates. The observations from the RabbitMQ management view or logs are **[TO BE PROVIDED]**.

6.10 Concurrency Testing

The dedicated concurrency test is: **"Two users attempt to book the same seat at the same time."** This test addresses the central problem of the project.

The expected behaviour is as follows:

- The first valid request obtains the Redis lock for the seat.
- The second request is rejected, receiving a conflict, while the seat is locked.
- The same seat must not be successfully booked by two users.

The method used to generate the simultaneous requests (for example, two browser sessions or two Postman requests sent together) and the observed outcome are **[TO BE PROVIDED]** and

[TO BE VERIFIED] respectively. The test is recorded as TC-11 in Section 6.11, and the evidence is placed in Figure 8.11.

6.11 Test Cases

The following test cases were designed from the functional requirements. The columns *Actual Result* and *Status* must be completed from real test execution and are therefore marked **[TO BE VERIFIED]**.

Table 6.2 Test cases for the Ticket Booking System

Test Case ID	Module	Test Scenario	Input	Expected Result	Actual Result	Status
TC-01	Registration	Register with valid details	Valid registration details	Account is created	**[TO BE VERIFIED]**	**[TO BE VERIFIED]**
TC-02	Login	Login with valid credentials	Registered user's credentials	User is logged in	**[TO BE VERIFIED]**	**[TO BE VERIFIED]**
TC-03	Login	Login with invalid credentials	Incorrect credentials	Login is rejected with a message	**[TO BE VERIFIED]**	**[TO BE VERIFIED]**
TC-04	Show browsing	View list of shows	Open show listing	Available shows are displayed	**[TO BE VERIFIED]**	**[TO BE VERIFIED]**
TC-05	Show browsing	Select date and time	Choose a show, date and time	Seat layout for the selected show is displayed	**[TO BE VERIFIED]**	**[TO BE VERIFIED]**
TC-06	Seat selection	View seat availability	Open seat page	Seat availability is displayed	**[TO BE VERIFIED]**	**[TO BE VERIFIED]**
TC-07	Seat selection	Select an available seat	One available seat	Redis lock is obtained; booking proceeds	**[TO BE VERIFIED]**	**[TO BE VERIFIED]**
TC-08	Seat selection	Select an already booked seat	A seat with status BOOKED	Selection is rejected	**[TO BE VERIFIED]**	**[TO BE VERIFIED]**
TC-09	Redis lock	Lock acquisition for first request	Seat selected by one user	Lock is created with a temporary expiry	**[TO BE VERIFIED]**	**[TO BE VERIFIED]**
TC-10	Redis lock	Lock expiry	Seat locked; booking not confirmed within expiry	Lock expires; seat becomes available again	**[TO BE VERIFIED]**	**[TO BE VERIFIED]**
TC-11	Concurrency	Two users attempt to book the same seat at the same time	Two simultaneous requests for one seat	First valid request obtains the lock; second request receives a conflict; seat not booked by both users	**[TO BE VERIFIED]**	**[TO BE VERIFIED]**
TC-12	Concurrency	Two users book different seats at the same time	Two simultaneous requests for different seats	Both requests obtain locks and proceed	**[TO BE VERIFIED]**	**[TO BE VERIFIED]**
TC-13	Booking	Confirm a booking	Locked seat and booking request	Booking is created and booking ID is generated	**[TO BE VERIFIED]**	**[TO BE VERIFIED]**
TC-14	Payment	Successful payment	Payment request for a booking	Booking confirmed; seat BOOKED; Redis lock released	**[TO BE VERIFIED]**	**[TO BE VERIFIED]**
TC-15	Payment	Unsuccessful payment	Failed payment request	Booking is not confirmed (behaviour **[TO BE PROVIDED]**)	**[TO BE VERIFIED]**	**[TO BE VERIFIED]**
TC-16	Ticket	View digital ticket	Open ticket for a confirmed booking	Booking ID and digital ticket are displayed	**[TO BE VERIFIED]**	**[TO BE VERIFIED]**

Test Case ID	Module	Test Scenario	Input	Expected Result	Actual Result	Status
TC-17	Booking history	View booking history	Open My Bookings	User's bookings are listed	**[TO BE VERIFIED]**	**[TO BE VERIFIED]**
TC-18	Cancellation	Cancel a booking	Confirmed booking	Booking status is updated and refund processing is initiated	**[TO BE VERIFIED]**	**[TO BE VERIFIED]**
TC-19	RabbitMQ	Booking request queued	Booking request	Message is queued and consumed	**[TO BE VERIFIED]**	**[TO BE VERIFIED]**
TC-20	RabbitMQ	Payment request queued	Payment request	Message is queued and consumed	**[TO BE VERIFIED]**	**[TO BE VERIFIED]**
TC-21	RabbitMQ	Refund request queued	Cancellation of a booking	Refund message is queued and consumed	**[TO BE VERIFIED]**	**[TO BE VERIFIED]**
TC-22	Database	Booking stored in MySQL	Confirmed booking	Booking record exists in the database	**[TO BE VERIFIED]**	**[TO BE VERIFIED]**
TC-23	Database	Seat status updated	Confirmed booking	Seat status is BOOKED	**[TO BE VERIFIED]**	**[TO BE VERIFIED]**
TC-24	API	Valid API requests in Postman	Valid requests for each API	Expected responses are returned	**[TO BE VERIFIED]**	**[TO BE VERIFIED]**
TC-25	API	Invalid API requests in Postman	Invalid or incomplete requests	Requests are rejected with an error response	**[TO BE VERIFIED]**	**[TO BE VERIFIED]**
TC-26	Admin	Administrator operation	**[TO BE PROVIDED]**	**[TO BE PROVIDED]**	**[TO BE VERIFIED]**	**[TO BE VERIFIED]**

CHAPTER 7 – RESULTS AND DISCUSSION

7.1 System Results

This chapter describes the results of the system in qualitative terms. No numerical performance values are given, because none were supplied. The statements in this chapter describe the behaviour defined by the design of the system, and each must be confirmed against the project's demonstrated output and the test records of Chapter 6 **[TO BE VERIFIED]**. The system provides the complete booking journey: registration and login, show browsing, date and time selection, real-time seat availability, seat selection, payment, booking ID and digital ticket, booking history and cancellation.

Table 7.1 Summary of results by module

Module	Result described	Evidence
Registration / login	Users can register and log in	Figures 8.1, 8.2 **[TO BE VERIFIED]**
Show browsing	Shows are listed with date and time selection	Figure 8.3 **[TO BE VERIFIED]**
Seat selection	Seat availability is displayed and seats can be selected	Figure 8.4 **[TO BE VERIFIED]**
Booking and payment	Booking is processed and a ticket is issued	Figures 8.5 to 8.7 **[TO BE VERIFIED]**
Booking history and cancellation	Bookings are listed and can be cancelled with refund processing	Figures 8.8, 8.9 **[TO BE VERIFIED]**
Redis concurrency	A second request for a locked seat receives a conflict	Figure 8.11 **[TO BE VERIFIED]**
RabbitMQ queues	Booking, payment and refund requests are queued	Figure 8.12 **[TO BE VERIFIED]**
Admin	**[TO BE PROVIDED]**	Figure 8.10 **[TO BE PROVIDED]**

7.2 User Registration/Login Results

A new user can register and log in, after which the booking functions are accessible. The login and registration pages are shown in Figures 8.1 and 8.2. Behaviour for invalid credentials is **[TO BE VERIFIED]**.

7.3 Show Browsing Results

The show listing presents the available shows, and the user can choose a date and time. Figure 8.3 shows the listing page **[TO BE VERIFIED]**.

7.4 Seat Selection Results

The seat selection page presents seat availability, and the user can select seats. A seat that is locked or booked by another user cannot be selected. Figure 8.4 shows this page **[TO BE VERIFIED]**.

7.5 Booking Results

A booking proceeds from seat selection through queue-based processing to confirmation. After confirmation the booking is stored in MySQL, the seat becomes BOOKED and the Redis lock is released (Figure 8.5) **[TO BE VERIFIED]**.

7.6 Payment Results

Payment is requested after seat selection and is handled through RabbitMQ; the outcome determines whether the booking is confirmed. Figure 8.6 shows the payment page. The nature of the payment (simulated or real) is **[TO BE PROVIDED]**.

7.7 Cancellation/Refund Results

A user can cancel a booking from the booking history. Cancellation initiates refund processing, and the status is shown to the user (Figures 8.8 and 8.9) **[TO BE VERIFIED]**.

7.8 Redis Concurrency Results

In the dedicated concurrency test, two users attempt to book the same seat at the same time. The expected and designed outcome is that the first valid request obtains the Redis lock and the second request receives a conflict, so that the seat is not successfully booked by both users. The observed outcome is **[TO BE VERIFIED]** and is shown in Figure 8.11.

7.9 RabbitMQ Queue Results

Booking, payment and refund-related requests are placed on RabbitMQ queues and processed. Evidence of queue processing is shown in Figure 8.12 **[TO BE VERIFIED]**.

7.10 Admin Results

The administrator dashboard is shown in Figure 8.10. The results of administrator operations are **[TO BE PROVIDED]**.

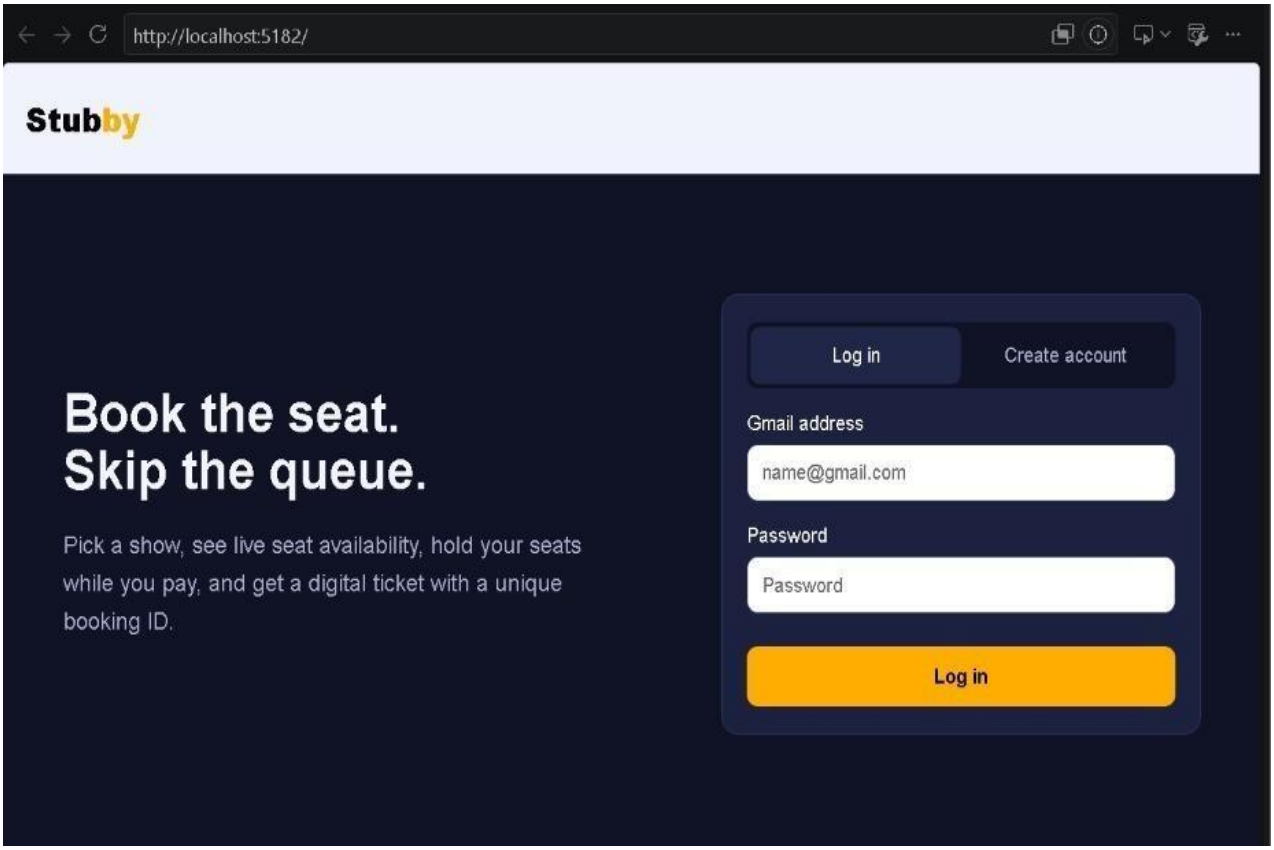
7.11 Discussion

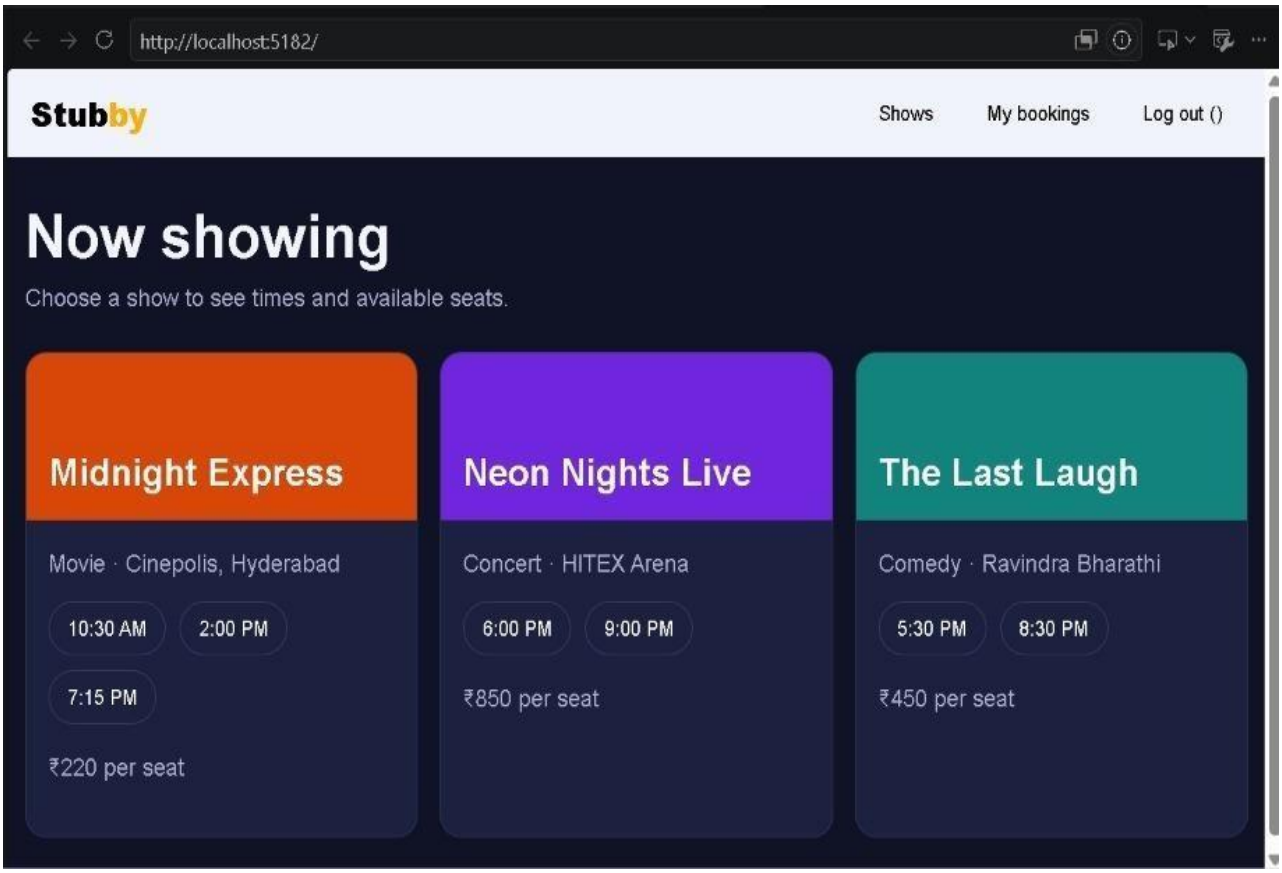
The results indicate that the combination of a Redis lock, a RabbitMQ queue and a MySQL database is a suitable way to approach the problem of concurrent seat booking. The lock is placed at the moment of seat selection, so conflicts are detected early and the second user is informed immediately. The temporary expiry prevents an abandoned selection from blocking a seat permanently. The queue separates the acceptance of a request from its processing, and MySQL retains the permanent record.

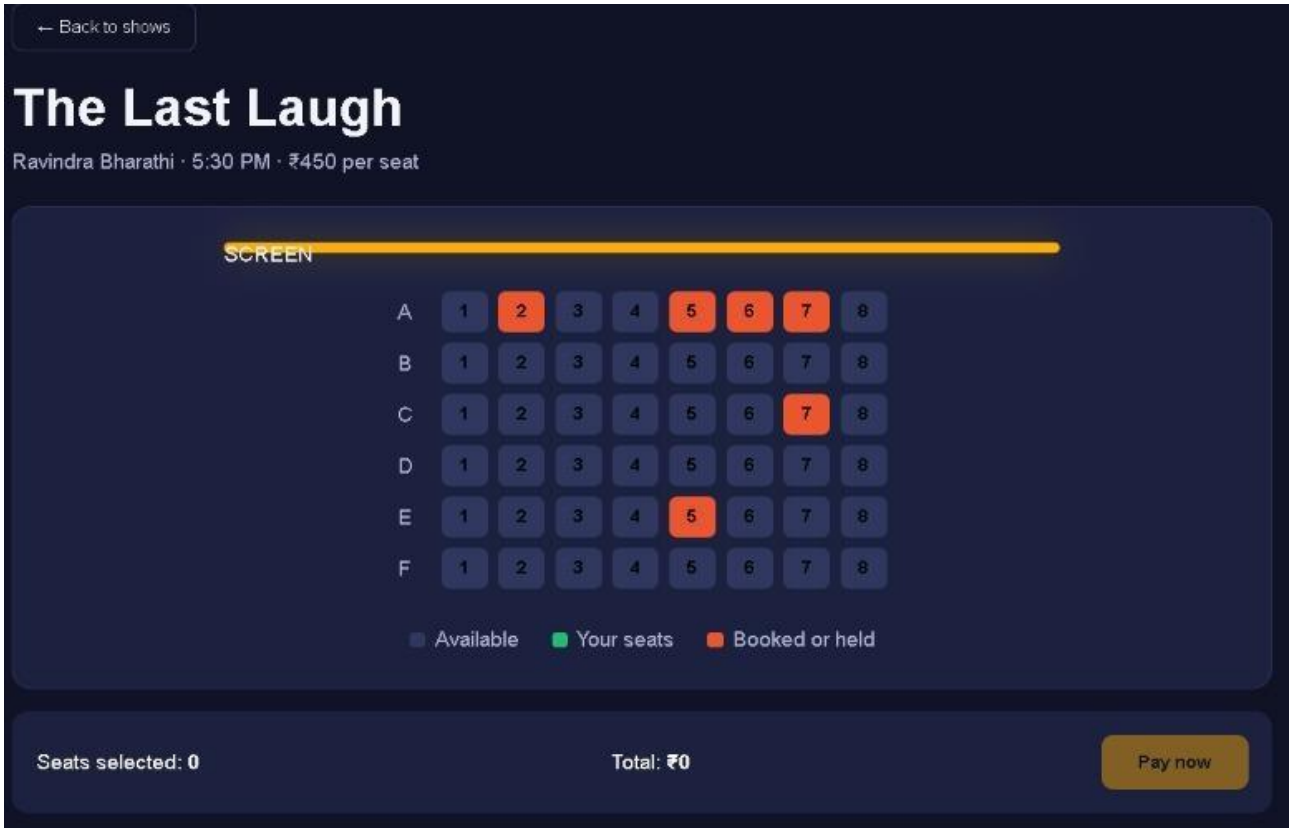
The discussion must also acknowledge limitations. The project has not been shown to be completely secure or unlimitedly scalable, and no load testing figures are reported. The lock expiry value, behaviour on payment failure, behaviour of a cancelled seat, and the extent of the administrator functions are not described in the material supplied and are **[TO BE PROVIDED]**. These points should be completed by the team so that the discussion reflects the actual implementation.

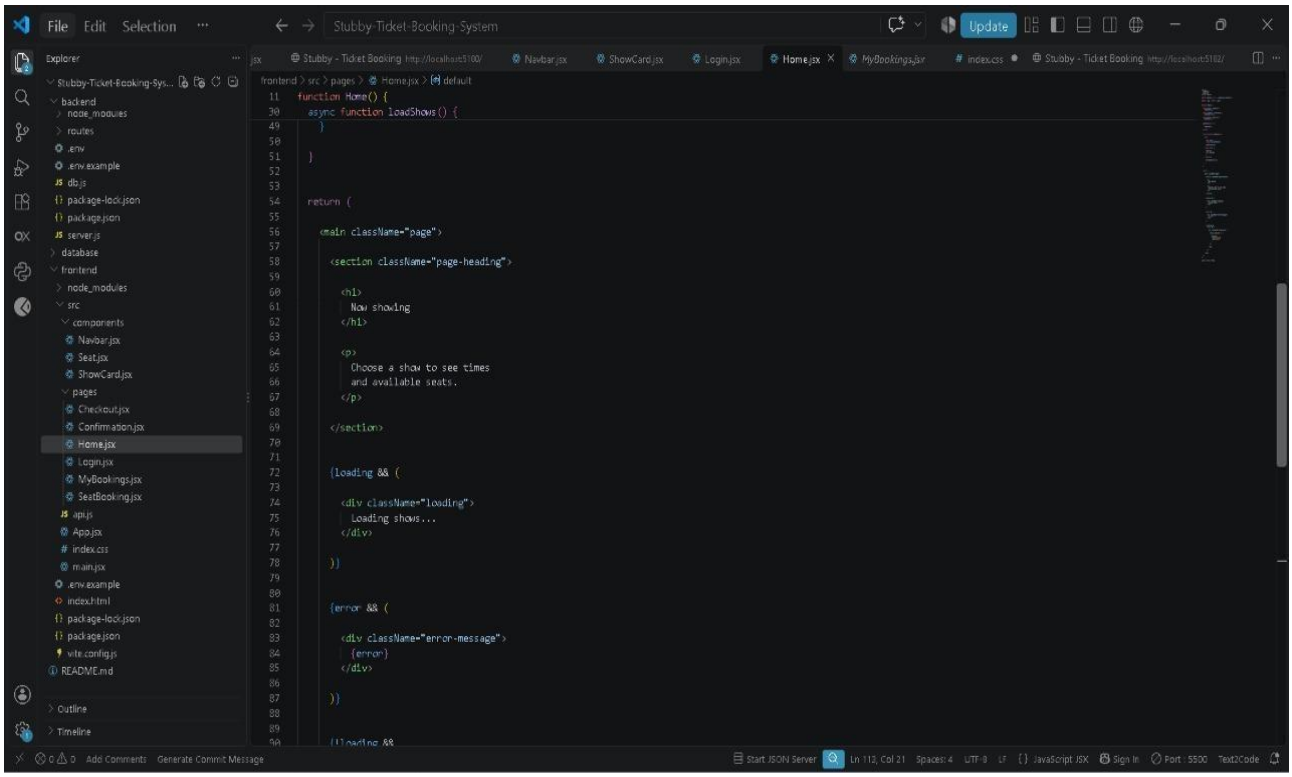
CHAPTER 8 – APPLICATION SCREENSHOTS

This chapter contains the screenshots of the application. Screenshots were not supplied with the project material; each place is therefore marked with the placeholder **[INSERT PROJECT SCREENSHOT HERE]** and is to be replaced with the corresponding screenshot.




http://localhost:5182/
Stubby
Shows
My bookings
Log out ()
Now showing
Choose a show to see times and available seats.
Midnight Express
Movie · Cinepolis, Hyderabad
10:30 AM
2:00 PM
7:15 PM
₹220 per seat
Neon Nights Live
Concert · HITEX Arena
6:00 PM
9:00 PM
₹850 per seat
The Last Laugh
Comedy · Ravindra Bharathi
5:30 PM
8:30 PM
₹450 per seat


← Back to shows
The Last Laugh
Ravindra Bharathi · 5:30 PM · ₹450 per seat
SCREEN
A 1 2 3 4 5 6 7 8
B 1 2 3 4 5 6 7 8
C 1 2 3 4 5 6 7 8
D 1 2 3 4 5 6 7 8
E 1 2 3 4 5 6 7 8
F 1 2 3 4 5 6 7 8
Available
Your seats
Booked or held
Seats selected: 0
Total: ₹0
Pay now


File Edit Selection
Stubby-Ticket-Booking-System
Update
Explorer
Stubby-Ticket-Booking-Sys...
backend
node_modules
routes
.env
.env.example
db.js
package-lock.json
package.json
server.js
database
frontend
node_modules
src
components
Navbar.jsx
Seat.jsx
ShowCard.jsx
pages
Checkout.jsx
Confirmation.jsx
Home.jsx
Login.jsx
MyBookings.jsx
SeatBooking.jsx
api.js
App.jsx
index.css
main.jsx
.env.example
index.html
package-lock.json
package.json
vite.config.js
README.md
Outline
Timeline
Navbar.jsx
ShowCard.jsx
Login.jsx
Home.jsx
MyBookings.jsx
index.css
frontend > src > pages > Home.jsx > default
function Home() {
async function loadShows() {
}
}
return (
<main className="page">
<section className="page-heading">
<h1>
Now showing
</h1>

Choose a show to see times
and available seats.

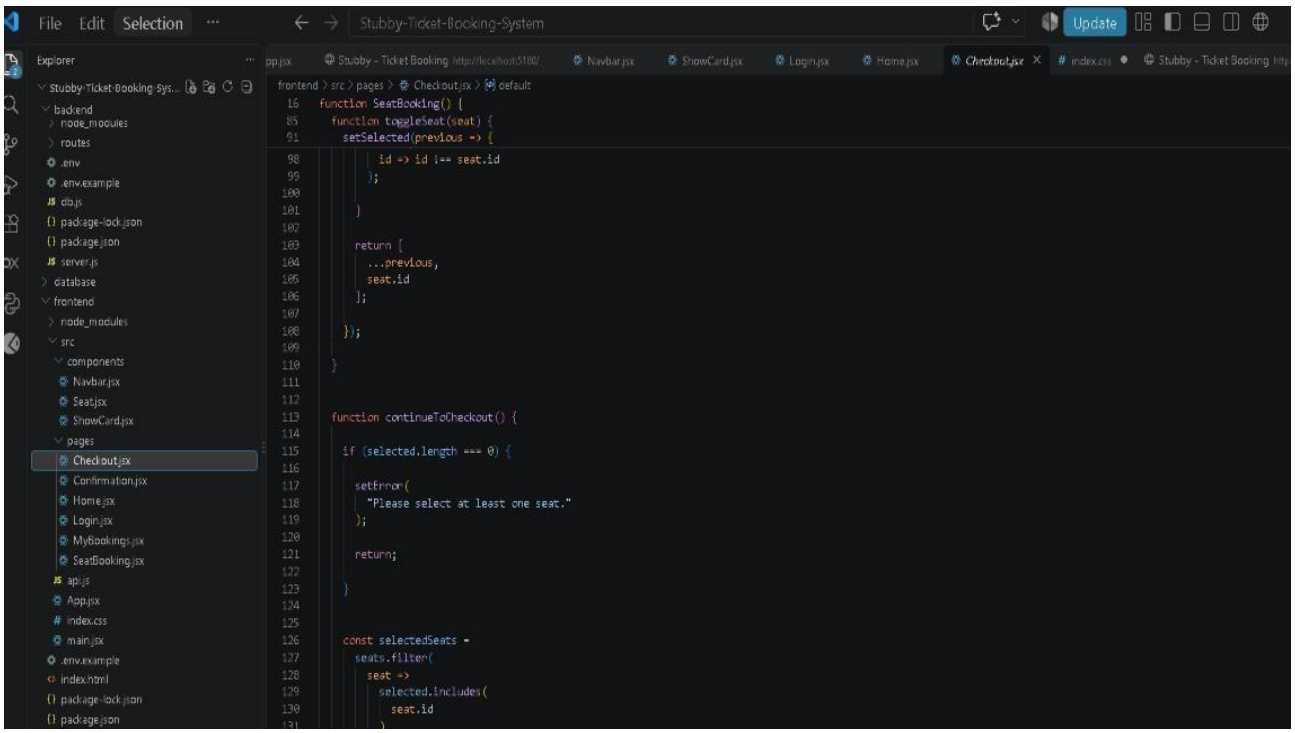
</section>
{loading && (

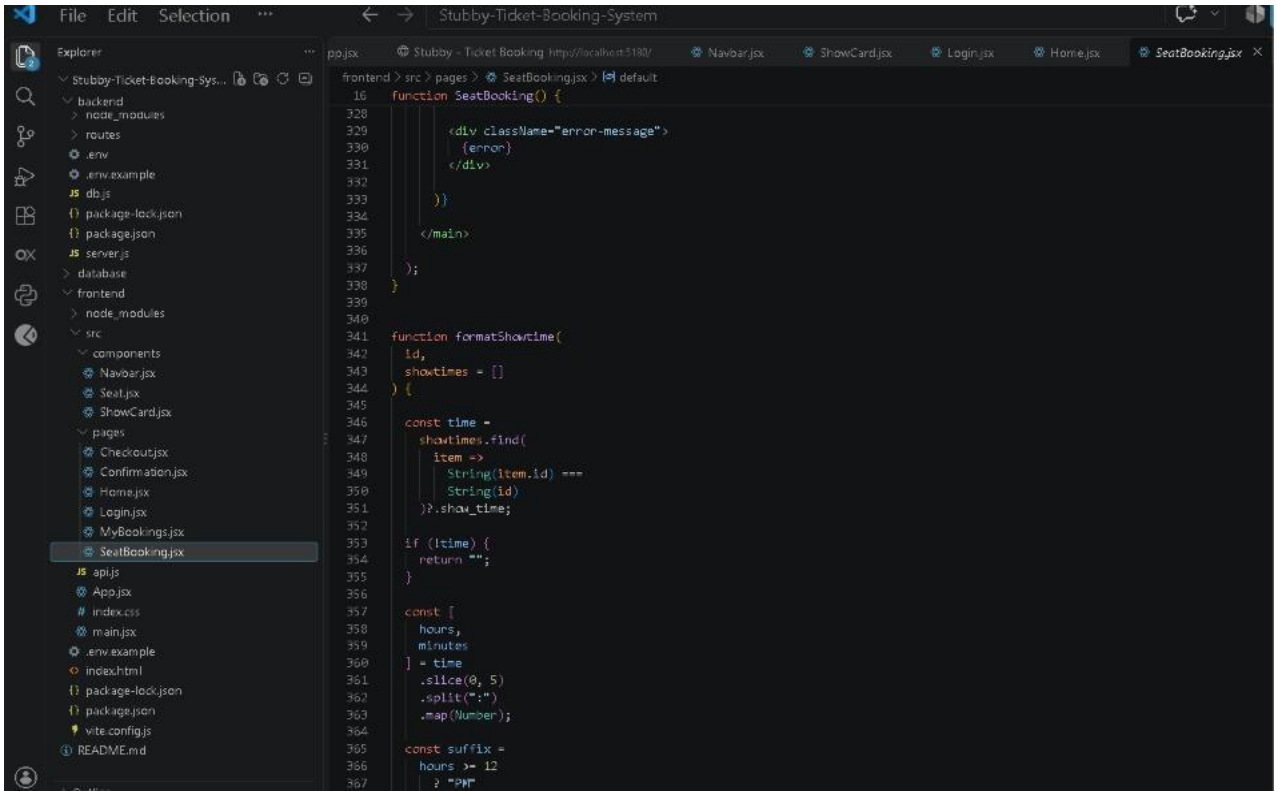
Loading shows...

)}
{error && (

{error}

)}
{!loading &&
Add Comments
Generate Commit Message
Start JSON Server
Ln 113, Col 21
Spaces: 4
UTF-8
LF
JavaScript JSX
Sign In
Port : 5500
Text2Code


File Edit Selection
Stubby-Ticket-Booking-System
Update
Explorer
Stubby-Ticket-Booking-Sys...
backend
node_modules
routes
.env
.env.example
db.js
package-lock.json
package.json
server.js
database
frontend
node_modules
src
components
Navbar.jsx
Seat.jsx
ShowCard.jsx
pages
Checkout.jsx
Confirmation.jsx
Home.jsx
Login.jsx
MyBookings.jsx
SeatBooking.jsx
api.js
App.jsx
index.css
main.jsx
.env.example
index.html
package-lock.json
package.json
Navbar.jsx
ShowCard.jsx
Login.jsx
Home.jsx
Checkout.jsx
index.css
frontend > src > pages > Checkout.jsx > default
function SeatBooking() {
function toggleSeat(seat) {
setSelected(previous => {
id => id !== seat.id
];
}
return [
...previous,
seat.id
];
});
}
function continueToCheckout() {
if (selected.length === 0) {
setError(
"Please select at least one seat."
);
return;
}
const selectedSeats =
seats.filter(
seat =>
selected.includes(
seat.id


File Edit Selection
Stubby-Ticket-Booking-System
Explorer
Stubby-Ticket-Booking-Sys...
backend
node_modules
routes
.env
.env.example
db.js
package-lock.json
package.json
server.js
database
frontend
node_modules
src
components
Navbar.jsx
Seat.jsx
ShowCard.jsx
pages
Checkout.jsx
Confirmation.jsx
Home.jsx
Login.jsx
MyBookings.jsx
SeatBooking.jsx
api.js
App.jsx
index.css
main.jsx
.env.example
index.html
package-lock.json
package.json
vite.config.js
README.md
pp.jsx
Stubby - Ticket Booking http://localhost:5180/
Navbar.jsx
ShowCard.jsx
Login.jsx
Home.jsx
SeatBooking.jsx
frontend > src > pages > SeatBooking.jsx > default
function SeatBooking() {

{error}

)}
</main>
);
}
function formatShowtime(
id,
showtimes = []
) {
const time =
showtimes.find(
item =>
String(item.id) ===
String(id)
)?.show_time;
if (!time) {
return "";
}
const [
hours,
minutes
] = time
.slice(0, 5)
.split(":")
.map(Number);
const suffix =
hours >= 12

Geo Tag photos with guide



CHAPTER 9 – CONCLUSION AND FUTURE SCOPE

9.1 Conclusion

This project presented the design and development of a web-based *Ticket Booking System* for movies, events and shows. The system lets a user register and log in, browse shows, select a date and time, view real-time seat availability, select seats, make payment, receive a booking ID and digital ticket, view booking history, and cancel bookings. Its main technical focus is concurrent seat booking. Redis is used for a temporary distributed seat lock, so that a second user attempting the same seat receives a conflict; RabbitMQ is used to queue booking, payment and refund-related requests; and MySQL stores users, shows, seats, bookings and their status. After a successful confirmation the seat becomes BOOKED and the lock is released.

The project demonstrates how concepts from database systems and distributed backend development can be combined in a practical application built with React.js, Java and Spring Boot. The conclusions are limited to what has been implemented and tested; verification of the results is recorded in Chapters 6 and 7.

9.2 Achievements

- Developed a web-based ticket booking application with a React.js frontend and a Java and Spring Boot backend.
- Implemented the booking journey from registration and login to digital ticket, booking history and cancellation.
- Applied Redis distributed seat locking with a temporary expiry to address concurrent seat booking.
- Used RabbitMQ to queue booking, payment and refund-related requests.
- Used MySQL to store users, shows, seats, bookings and related status information.
- Designed a dedicated concurrency test in which two users attempt to book the same seat at the same time.
- Distributed the work among team members according to backend and database, frontend, and concurrency and integration roles.

9.3 Limitations

- Real payment gateway integration is not part of the present system.
- Email and SMS notifications are not part of the present system.
- The system is not deployed on a cloud platform, and no mobile application is provided.
- Support for multiple theatres is not part of the present system.

- Security is not claimed to be complete; stronger security is future scope.
- No measured performance or load-testing values are reported **[TO BE PROVIDED]**.
- The expiry of the seat lock is temporary, so a user must complete the booking within that period.

9.4 Future Scope

The following enhancements are identified as future scope and are not part of the present implementation.

- **Android/iOS mobile application:** a mobile client would allow users to book tickets from their phones.
- **Cloud deployment:** hosting the application, database, Redis and RabbitMQ on a cloud platform would make it available beyond a local environment.
- **Real payment gateway:** integration with a real payment gateway would replace the present payment step.
- **Email/SMS notifications:** confirmation and cancellation messages could be sent to users.
- **Analytics and AI recommendations:** booking data could be analysed to provide insights and recommend shows to users.
- **Multiple theatres:** the system could be extended to support more than one theatre.
- **Stronger security:** additional security measures could be introduced to protect user and payment data.

REFERENCES

No references were included in the project material supplied. The entries below are placeholders and must be replaced with the actual sources used by the team (for example books, papers, official documentation or websites). No reference has been invented.

[1] **[REFERENCE TO BE ADDED]**

[2] **[REFERENCE TO BE ADDED]**

[3] **[REFERENCE TO BE ADDED]**

[4] **[REFERENCE TO BE ADDED]**

[5] **[REFERENCE TO BE ADDED]**

[6] **[REFERENCE TO BE ADDED]**

[7] **[REFERENCE TO BE ADDED]**

[8] **[REFERENCE TO BE ADDED]**

APPENDICES

Appendix A – Important Source Code

The source code excerpts of the project are **[TO BE PROVIDED]**. The following items are suggested for inclusion, once supplied by the team:

- Redis seat lock acquisition and release (concurrency and integration code).
- RabbitMQ producer and consumer for booking, payment and refund requests.
- Booking confirmation logic that updates MySQL and releases the lock.
- Selected REST controller methods.
- Selected React components for seat selection and booking.

Appendix B – API Endpoints

Table 9.1 API endpoints

Sl. No.	Operation	Method	URL	Description
1	Register user	**[TO BE PROVIDED]**	**[TO BE PROVIDED]**	**[TO BE PROVIDED]**
2	Login	**[TO BE PROVIDED]**	**[TO BE PROVIDED]**	**[TO BE PROVIDED]**
3	List shows	**[TO BE PROVIDED]**	**[TO BE PROVIDED]**	**[TO BE PROVIDED]**
4	Seat availability	**[TO BE PROVIDED]**	**[TO BE PROVIDED]**	**[TO BE PROVIDED]**
5	Lock / select seat	**[TO BE PROVIDED]**	**[TO BE PROVIDED]**	**[TO BE PROVIDED]**
6	Booking and payment	**[TO BE PROVIDED]**	**[TO BE PROVIDED]**	**[TO BE PROVIDED]**
7	Booking history	**[TO BE PROVIDED]**	**[TO BE PROVIDED]**	**[TO BE PROVIDED]**
8	Cancel booking	**[TO BE PROVIDED]**	**[TO BE PROVIDED]**	**[TO BE PROVIDED]**

Appendix C – Database Tables

Table 9.2 Users table structure

Column	Data Type	Key / Constraint	Description
[TO BE PROVIDED]	**[TO BE PROVIDED]**	**[TO BE PROVIDED]**	**[TO BE PROVIDED]**

Table 9.3 Shows table structure

Column	Data Type	Key / Constraint	Description
[TO BE PROVIDED]	[TO BE PROVIDED]	[TO BE PROVIDED]	[TO BE PROVIDED]

Table 9.4 Seats table structure

Column	Data Type	Key / Constraint	Description
[TO BE PROVIDED]	[TO BE PROVIDED]	[TO BE PROVIDED]	[TO BE PROVIDED]

Table 9.5 Bookings table structure

Column	Data Type	Key / Constraint	Description
[TO BE PROVIDED]	[TO BE PROVIDED]	[TO BE PROVIDED]	[TO BE PROVIDED]

Appendix D – Test Cases

The complete list of test cases is given in Table 6.2 (Section 6.11). The table below is a record sheet for the execution of those test cases.

Table 9.6 Test execution record

Test Case ID	Date of execution	Tested by	Evidence (figure / log)
TC-07	[TO BE PROVIDED]	[TO BE PROVIDED]	[TO BE PROVIDED]
TC-11	[TO BE PROVIDED]	[TO BE PROVIDED]	[TO BE PROVIDED]
TC-14	[TO BE PROVIDED]	[TO BE PROVIDED]	[TO BE PROVIDED]
TC-18	[TO BE PROVIDED]	[TO BE PROVIDED]	[TO BE PROVIDED]
TC-19	[TO BE PROVIDED]	[TO BE PROVIDED]	[TO BE PROVIDED]